Deep Learning & Generative AI

Beginner study guide: plain-language notes + runnable experiments with outputs

Part 1: notes (01 history · 02 AI/ML/DL/GenAI & the neuron · 03 perceptron · 04 activation functions · 05 normalization & regularization)
Part 2: notebooks (each lecture's claims tested in code)

Plus the cheat sheet. Runnable notebooks are in the **notebooks/** folder next to this PDF.

01 · History of deep learning (lecture of 04-08-2026)

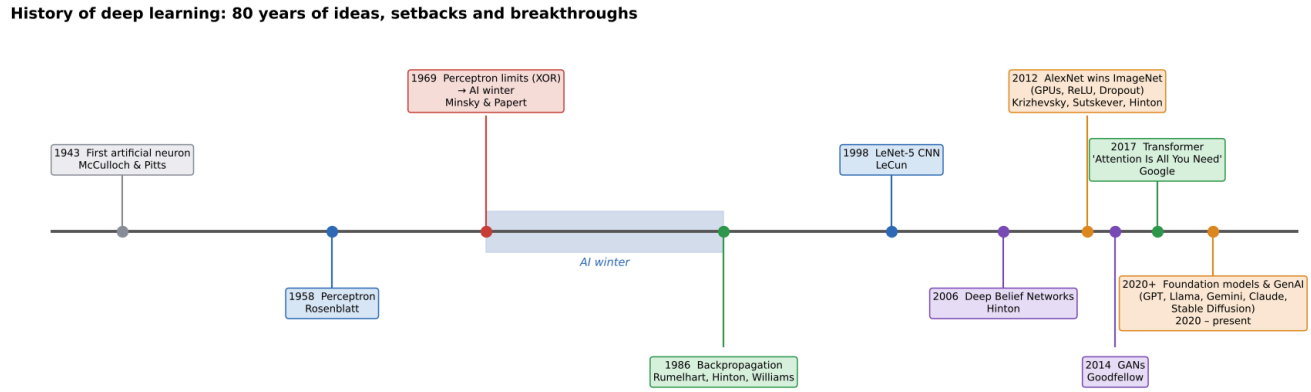
Notebook: `notebooks/02_neurons_1943_to_1986.ipynb` runs the first four milestones (1943 → 1986) as code.

---

1. What is deep learning?

> **Deep learning** is a branch of machine learning that uses **artificial neural networks with multiple hidden layers** to learn complex patterns from data.

It became famous only in the last decade or so, but its roots go back to the **1940s**. The field moved through **ideas, setbacks and breakthroughs** before becoming the engine of today's AI.



---

2. The ten milestones (learn the year, the people and the one-line contribution)

Year	Milestone	Who	Why it matters
1943	first **artificial neuron**	Warren **McCulloch** & Walter **Pitts**	first mathematical model of a biological neuron; showed that neurons with binary inputs and outputs can do **logic** (AND, OR, NOT); brain functions can be represented **computationally**
1958	**Perceptron**	Frank **Rosenblatt**	the first **trainable** neural network: it learns its weights from examples. Single layer, supervised, binary classification. **Limitation: only linearly separable problems**
1969	***Perceptrons*** (the book)	Marvin **Minsky** & Seymour **Papert**	showed the limits of single-layer networks (e.g. **XOR**) → funding fell, research moved to rule-based **expert systems** → the **AI winter**

Year	Milestone	Who	Why it matters
1986	**Backpropagation** popularised	David **Rumelhart**, Geoffrey **Hinton**, Ronald **Williams**	trains **multi-layer** networks by sending the error **backwards**, so deeper networks can learn complex patterns. Still **the** standard training algorithm
1998	**LeNet-5** (CNN)	Yann **LeCun**	one of the first successful **convolutional** networks: handwritten digits, bank cheques, postcodes. Showed networks can handle **images**
2006	**Deep Belief Networks**	Geoffrey **Hinton**	**layer-wise pre-training** made deeper networks practical. Marks the **start of modern deep learning**
2012	**AlexNet** wins **ImageNet**	Alex **Krizhevsky**, Ilya **Sutskever**, Geoffrey **Hinton**	trained on **GPUs**, used **ReLU** and **Dropout**, and crushed traditional computer-vision methods. Sparked wide adoption in academia and industry
2014	**GANs**	Ian **Goodfellow**	**generator vs discriminator**: image generation, face synthesis, super-resolution, data augmentation. Opened up **generative AI**
2017	**Transformer**	researchers at **Google** ("**Attention Is All You Need**")	built on the **attention** mechanism, replaced recurrent networks for language: translation, chatbots, summarisation, code generation
2020–now	**Foundation models & Generative AI**	many organisations	**LLMs**, multimodal AI, **diffusion models**, AI assistants: GPT, Llama, Gemini, Claude, Stable Diffusion

Memory trick: three "Hinton" years (1986 backprop, 2006 DBNs, 2012 AlexNet). He's often called a "godfather of deep learning".

Two stories worth telling in an exam answer

- **The AI winter (1969 → mid-1980s).** A single perceptron draws one straight line, so it can't learn **XOR** (the 1s are on one diagonal, the 0s on the other). Minsky and Papert's book made this famous, funding for neural networks dried up, and AI research turned to hand-written rules (expert systems). **Backpropagation (1986)** ended it: multi-layer networks *can* learn XOR, and now they could be **trained**. Notebook 02 reproduces the whole story in code.
- **The 2012 turning point.** The ideas (CNNs, backprop) already existed. What changed was **big data** (ImageNet's 1.2 million labelled images), **GPUs** (fast parallel training) and **better tricks** (ReLU, Dropout). AlexNet's win convinced the world.

---

3. Why deep learning succeeded (the five enabling factors)

Factor	Contribution
large datasets	more data → better training (ImageNet, web text)

Factor	Contribution
GPUs and TPUs	massively parallel hardware made training deep networks fast enough
improved algorithms	backpropagation, **ReLU**, **Batch Normalization**, **Dropout**, **attention** (chapters 04–05)
open-source frameworks	TensorFlow, **PyTorch**, Keras made building networks accessible
cloud computing	scalable compute to train very large models

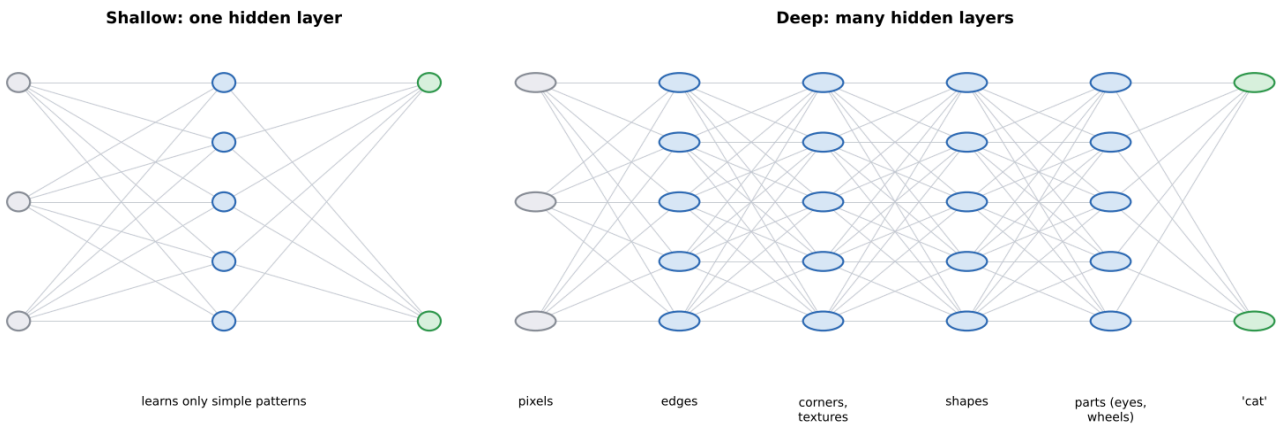
4. How the models evolved

artificial neuron → perceptron → multi-layer perceptron (MLP) → CNN → RNN → LSTM → Transformer → foundation models and GenAI

- **MLP:** hidden layers (solves XOR).
- **CNN:** images, with local filters.
- **RNN / LSTM:** sequences, with memory (the LSTM remembers longer).
- **Transformer:** attention, parallel training, the basis of LLMs.

---

5. Why is it called "deep" learning?



> "Deep" refers to the **number of hidden layers** between input and output: the **depth** of the network. It does **not** mean deep thinking, deep intelligence or deep mathematics.

Each layer builds on the previous one (hierarchical feature learning). The lecture's example, recognising a cat or an object:

Layer	Learns
input	raw pixels
layer 1	edges
layer 2	corners and textures
layer 3	shapes
layer 4	object parts (eyes, nose, wheels)

Layer	Learns
layer 5	the complete object

The lecture's face-recognition version: pixels → edges and lines → eyes, nose, mouth → face shape → identity.

The key point: nobody programs these features. **The network learns them automatically** from data. That's the big difference from classical machine learning, where a human designs the features (chapter 02, and notebook 01 shows it).

A shallow network (one hidden layer) learns only simple features. **A deep network** builds increasingly **abstract** representations, which is why deep learning leads in computer vision, speech recognition, natural-language processing and generative AI.

Key takeaways (from the lecture)

- Deep learning started with the **artificial neuron in 1943**.
- The **perceptron (1958)** made networks trainable but was limited to simple (linearly separable) problems.
- **Backpropagation (1986)** made multi-layer networks trainable.
- **CNNs (1998)** transformed computer vision, and **AlexNet (2012)** showed the power of deep networks on GPUs.
- **Transformers (2017)** revolutionised NLP and underpin modern **LLMs**.
- Today deep learning powers generative AI, autonomous systems, healthcare, finance, robotics and science.

Check yourself

1. Who built the first artificial neuron, and when? What could it do?
2. What did Rosenblatt's perceptron add, and what was its limitation?
3. What caused the AI winter, and what ended it?
4. What three ingredients did AlexNet combine in 2012?
5. Which 2017 paper introduced the Transformer, and what mechanism is it built on?
6. What does "deep" in deep learning refer to? What does it NOT mean?
7. Name the five factors that enabled deep learning's success.
8. Put in order: Transformer, LSTM, perceptron, CNN, MLP, artificial neuron, RNN.

Answers

1. McCulloch and Pitts, 1943. A mathematical model of a neuron that performed logical operations with binary inputs and outputs.
2. Learning: it was the first trainable neural network, adjusting its weights from examples. It could only solve linearly separable problems (not XOR).
3. Minsky and Papert's *Perceptrons* (1969) exposed the limits of single-layer networks, and funding moved to rule-based AI. Backpropagation (1986) made multi-layer networks trainable.
4. GPUs, ReLU and Dropout (plus the big ImageNet dataset).
5. "Attention Is All You Need" (Google). The attention mechanism.

6. The number of hidden layers (the depth). Not deep thinking, deep intelligence or deep mathematics.
7. Large datasets, GPUs and TPUs, improved algorithms, open-source frameworks, cloud computing.
8. Artificial neuron → perceptron → MLP → CNN → RNN → LSTM → Transformer.

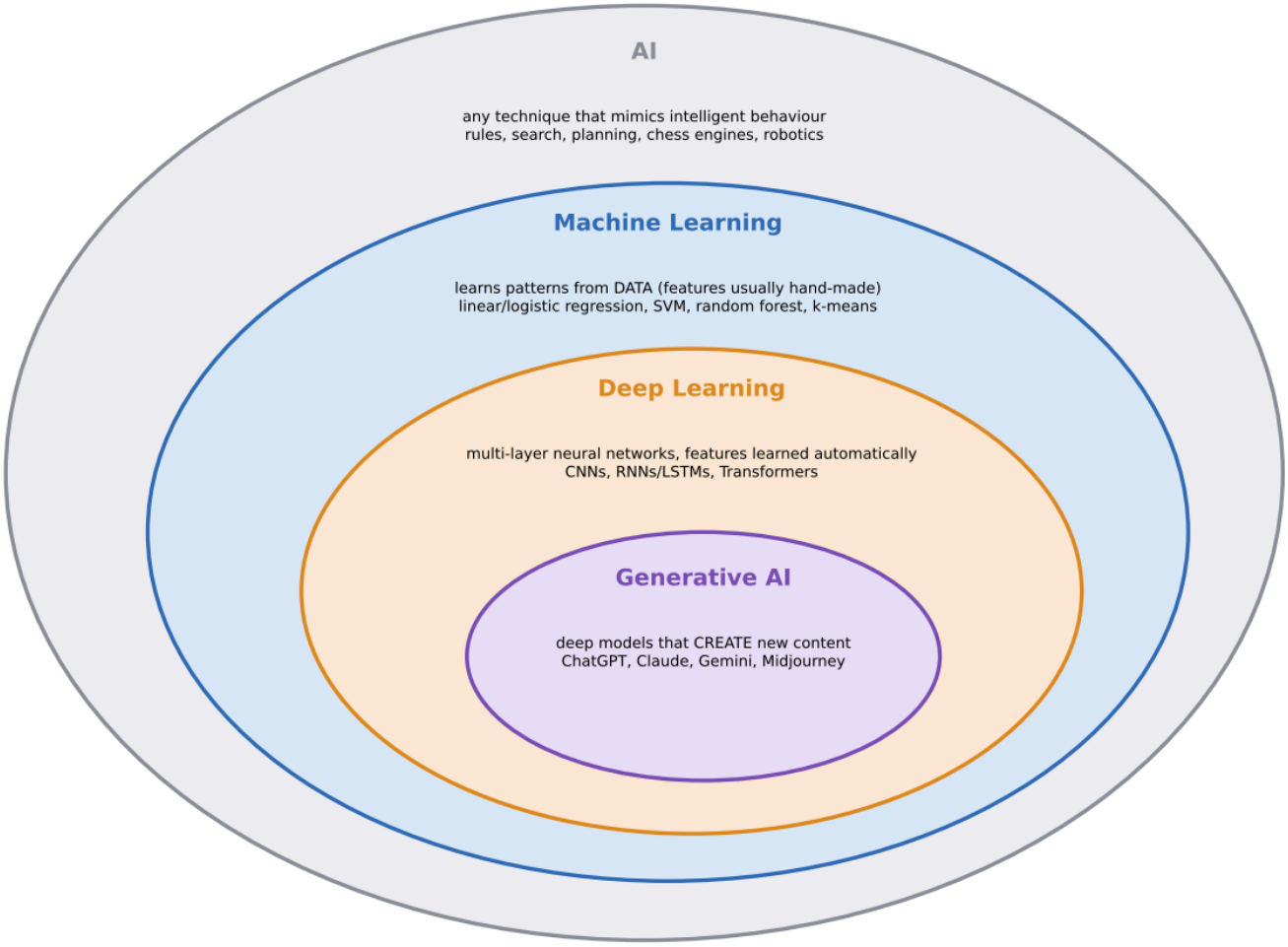
02 · AI vs ML vs DL vs GenAI, and the biological neuron analogy (lecture of 05-08-2026)

Notebooks: - `notebooks/01_ai_ml_dl_genai_in_code.ipynb` : one problem solved at all four levels; - `notebooks/02_neurons_1943_to_1986.ipynb` : the neuron, from biology to code.

Part A · AI vs ML vs DL vs GenAI

1. The hierarchy: nested circles

AI ⊃ ML ⊃ DL ⊃ GenAI: each circle is a special case of the one around it



ML is a subset of AI, DL is a subset of ML, GenAI is a subset (and an application) of DL. Each inner circle keeps the properties of the one around it and adds its own defining constraint.

Level	Defining idea	Examples
Artificial Intelligence (AI)	any technique that lets a machine **mimic intelligent behaviour**	rule-based expert systems, search algorithms, robotics, chess engines
Machine Learning (ML)	AI that **learns patterns from data** instead of following hand-written rules	random forest, SVM, logistic regression
Deep Learning (DL)	ML using **multi-layer neural networks** that **learn features automatically** from raw data	CNNs, RNNs, Transformers

Level	Defining idea	Examples
Generative AI (GenAI)	DL models trained to **generate new content** (text, images, audio, code), not just classify or predict	ChatGPT, Claude, Gemini, Midjourney, DALL-E

2. Each level in more detail

AI: the broadest term

- Any system doing tasks that normally need human intelligence: reasoning, planning, perception, language, decision-making.
- **Not defined by a technique.** It includes 1960s–80s **symbolic, rule-based** systems (if-then medical-diagnosis rules), **search and planning** (pathfinding, early chess programs), all of ML, and **robotics and control**.
- Best seen as a **goal / field of study**: building machines that behave intelligently. ML, DL and GenAI are **strategies** for reaching that goal.

ML: learning from data

- Instead of a person writing the logic, the algorithm gets **examples** and a learning procedure and **infers** the logic.
- It learns an input → output mapping (or finds structure in unlabelled data), and usually **improves with more data**.
- It **often still needs a human to engineer the features** (which input variables the model sees).
- Three paradigms: **supervised, unsupervised, reinforcement** learning.

Algorithm	Typical use
linear / logistic regression	a number or a yes/no from tabular data
decision trees / random forests	classification and regression on business data
SVM	classification with a clear margin
k-means	grouping unlabelled data (customer segments)

DL: deep neural networks

- **Many layers** ("deep"), each turning its input into a **more abstract representation**: edges → shapes and textures → whole objects or faces. This **hierarchical feature extraction** is the mechanical difference from classical ML.
- **Automatic feature extraction** from raw data, with no manual feature engineering.
- Needs **much more data and compute** (GPUs).
- **Excels at unstructured data:** images, audio, video, text.
- Architectures: **CNNs** (images), **RNNs/LSTMs/GRUs** (sequences), **Transformers** (language and beyond).

GenAI: creating new content

- A classifier says "cat or dog". A generative model **draws a new cat that never existed**.
- Trained, often **self-supervised**, on **massive** data to learn the **distribution** of the data, not just a decision boundary.
- Produces new text, images, audio, video and code.
- Mostly **Transformer**-based (LLMs), plus **diffusion models** for images and video.
- What separates it from older generative methods is mainly **scale**: data, parameters, compute.

System	Generates
ChatGPT, Claude, Gemini	text: conversation, writing, code, analysis
Midjourney, DALL-E	images from text
Runway	video generation and editing
ElevenLabs	synthetic speech, voice cloning

3. The comparison table (exam favourite)

	AI	ML	DL	GenAI
scope	broadest	subset of AI	subset of ML	subset of DL
core mechanism	rules, search, learning…	statistical patterns from examples	multi-layer networks, automatic features	large networks that model and reproduce the data distribution
feature engineering	n/a / varies	**usually manual**	**automatic**	automatic, at massive scale
typical data needs	varies	**moderate** (hundreds to thousands)	**large** (100k+)	**massive** (web-scale, billions of tokens or images)
typical output	any intelligent behaviour	prediction / classification	prediction, classification, detection	**newly generated content**
example	chess program	spam filter	CNN finding tumours on X-rays	LLM writing an essay

4. The three distinguishing factors

1. **Data.** The amount needed rises sharply from ML → DL → GenAI.
2. Classical ML: often fine with thousands of rows of tabular data.
3. DL: needs large labelled sets (or augmentation) to avoid **overfitting** its many parameters.
4. GenAI: **self-supervised** learning on web-scale, mostly unlabelled data.
5. **Feature engineering.**
6. ML: a human selects and builds features (e.g. "average transaction in the last 30 days" for fraud detection).
7. DL: learned automatically by the layers.
8. GenAI: learned at huge scale, producing rich **embeddings** that transfer to many tasks.
9. **Architecture.**

10. ML: simple, well-understood models (trees, SVMs, linear functions).
11. DL: **artificial neural networks** (MLPs, CNNs, RNNs/LSTMs/GRUs).
12. GenAI: overwhelmingly **Transformers** (self-attention relates all parts of the input at once), plus **diffusion models**.

5. Real systems mapped (shows the nesting)

System	AI	ML	DL	GenAI
1980s chess engine with hand-coded rules	✓	✗	✗	✗
Naive Bayes spam filter	✓	✓	✗	✗
CNN photo tagger ("beach", "mountain")	✓	✓	✓	✗
ChatGPT writing a poem	✓	✓	✓	✓

Every GenAI system is also DL, ML and AI, **but not the other way round**.

6. Common misconceptions

- **"AI and ML are the same."** ML is one way to build AI, and many AI systems (rule-based) don't learn at all.
- **"DL is just ML with more layers."** Technically true, but the real shift is **automatic feature learning**, which changes the whole workflow.
- **"GenAI is a different technology from DL."** No: it is deep learning (mostly Transformers) applied and scaled to content generation.
- **"Bigger is always better."** A well-tuned classical model can beat a big deep network on **small, structured** data.

Why it matters in practice: - a few thousand rows of business data → **classical ML**; - lots of labelled images → **DL (CNNs)**; - producing new content (a summary, code, an image) → **GenAI**.

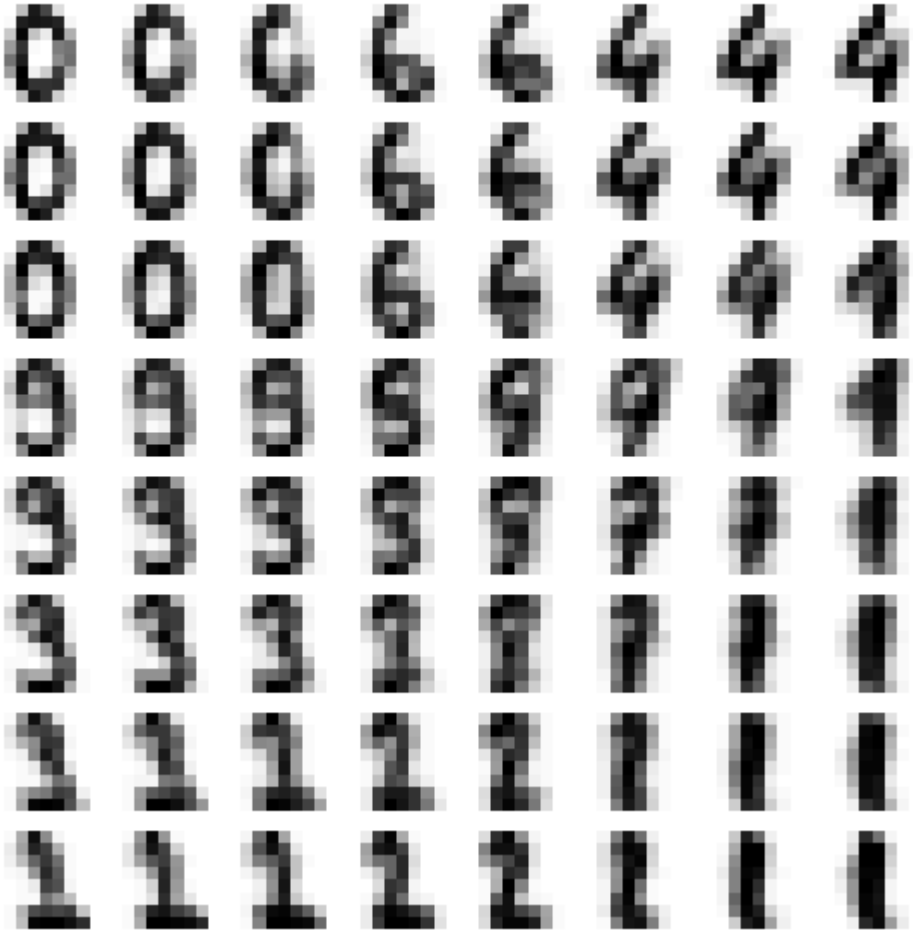
Picking the right category is often the most important early decision in an AI project.

What notebook 01 measured (same digits, four levels)

Level	Approach	Result
AI	hand-written if-then rules	97% on **0 vs 1 only**; rules for all 10 digits are impractical
ML	logistic regression on 8 **hand-engineered** features	74% on all 10 digits; limited by the features chosen
ML	the same model on 64 raw pixels	97%
DL	2-hidden-layer network on raw pixels	98%, with features it invented itself
GenAI	a tiny generative network (VAE)	**draws new digits** that aren't copies of training images

Honest note: on these tiny, clean 8×8 digits, the deep network only tied with logistic regression on raw pixels at every training-set size. DL's big advantage appears on large, complex data (photos, speech, text).

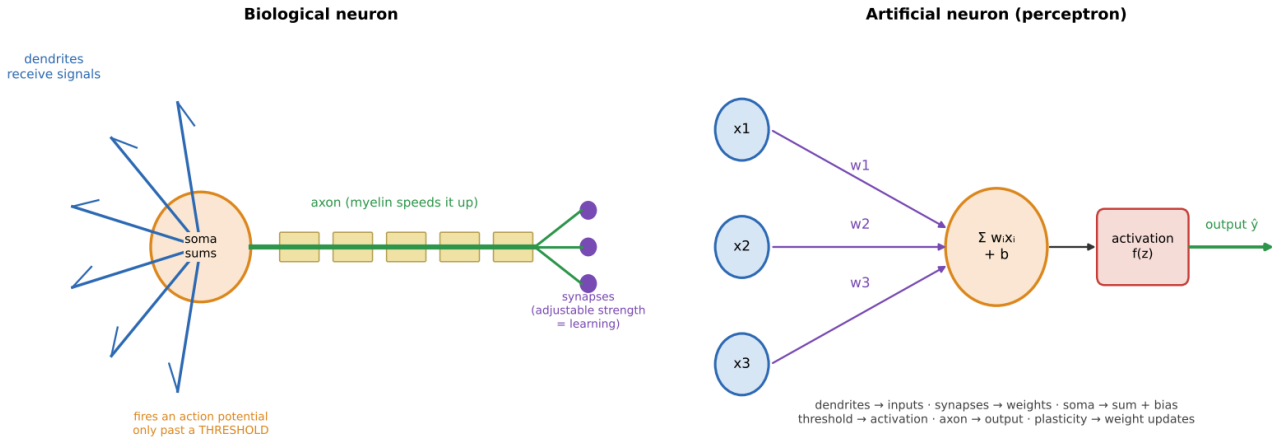
64 NEW digits generated from a grid of 2-number codes (none of them is a copy of a training image)



Part B · The biological neuron analogy

7. Why the brain?

Artificial neural networks were **directly inspired by biological neurons**. The **McCulloch–Pitts neuron (1943)** and the **perceptron (1958)** were attempts to model, mathematically, how a neuron receives, processes and passes on information. Modern DL has moved far from biology, but its vocabulary still maps onto it.



8. The biological neuron (four parts)

The human brain has about **86 billion neurons**, each connected to thousands of others.

Part	What it does
dendrites	branch-like **inputs**: receive signals from other neurons (thousands of them)
soma (cell body)	**integrates (sums)** the incoming signals. If the total passes a **threshold**, it fires an **action potential**, otherwise nothing: **all-or-nothing**
axon	long cable carrying the output signal to other neurons (under 1 mm to over 1 m long). **Myelin** insulation speeds it up
synapse	the junction with the next neuron. The signal crosses a tiny gap (**synaptic cleft**) chemically, via **neurotransmitters**. Its **strength can change** with experience (**synaptic plasticity**), the biological basis of **learning and memory**

Signal path: dendrites receive → soma sums (weighted by synapse strengths) → fires if over the threshold → action potential travels down the axon → converted to chemicals at the synapse → influences the next neuron. Synapse strengths change over time, which is how the network learns.

9. The artificial neuron (perceptron)

A **drastic mathematical simplification** that keeps the essential computation: 1. **Inputs** x_1, x_2, x_3… are the feature values (pixels, word embeddings). They play the role of the **dendrites**. 2. **Weights** w_1, w_2, w_3… set the importance of each input, like **synapse strength**. Training

adjusts them (backpropagation + gradient descent), the analogue of **plasticity**. 3. **Summation:** $z = \Sigma\, w_i x_i + \mathbf{b}$ (the bias shifts the sum), like the **soma**. 4. **Activation function** f(z): the **threshold** idea, adding non-linearity (step, sigmoid, ReLU, tanh). **Without it, any number of layers collapses into one linear function** (chapter 04 proves this in code).

10. The mapping table (learn it)

Biological	Artificial	Role
dendrites	inputs x_1, x_2, …	receive signals / feature values
synapse (strength)	weights w_1, w_2, …	importance of each signal
soma	$\Sigma\, w_i x_i + b$	integrate into one value
firing threshold / action potential	activation function	whether, and how strongly, to "fire"
axon	output to the next layer	carry the result forward
synaptic plasticity	weight updates (backpropagation)	learn from experience / data

From one neuron to a network

Brain	Artificial neural network
billions of neurons in regions and circuits (visual cortex, motor cortex)	layers: input, hidden, output
signals pass through many layers of neurons	**forward propagation** through the layers
learning = strengthening/weakening synapses	learning = **backpropagation** weight updates
regions specialise (vision, language)	architectures specialise (CNNs for images, RNNs/Transformers for sequences)

11. Similarities and where the analogy breaks

Similar: - both take **many weighted inputs**, **sum** them, and apply a **threshold / non-linearity**; - both **adjust connection strengths** with experience; - both get intelligence only from **many connected units**, never from one.

Different (don't treat ANNs as brain models):

	Biological	Artificial
signal	electrochemical (ion channels, neurotransmitters)	numbers (floating point)
timing	asynchronous, continuous-time spikes	synchronous, discrete steps
learning	biochemical plasticity, not fully understood	an exact procedure (backprop + gradient descent)
complexity	thousands of synapses, many neuron types, glial support	weighted sum + activation
energy	**~20 watts** for the whole brain	training big models uses enormous energy

	Biological	Artificial
scale	~86 billion neurons, **~100 trillion synapses**	even the largest models have fewer parameters than the brain has synapses

The artificial neuron is **inspired by**, but a **drastic simplification** of, the real one, **by design**: the goal is engineering performance, not biological accuracy.

12. Why the analogy is still useful

- **Many layers:** the brain processes information hierarchically across regions, and deep networks build abstraction layer by layer.
- **Learned weights:** synapses adapt with experience, and weights are what training adjusts.
- **Activation functions:** neurons fire only past a threshold, and networks need non-linearity.
- **More connections → more complex behaviour**, up to a point.

Connecting the two halves: DL (Part A) is *defined* by artificial neural networks, which are built from the neurons in Part B. Networks of millions or billions of simple neurons need a correspondingly huge amount of data to tune every connection, which is why DL and GenAI need so much more data and compute than classical ML.

Check yourself

1. Draw the hierarchy. Where does ChatGPT sit? A rule-based chess program?
2. Name the three factors that distinguish ML, DL and GenAI.
3. Who chooses the features in classical ML? In DL?
4. Give two misconceptions about AI/ML/DL/GenAI and correct them.
5. Map each part: dendrites, synapse, soma, threshold, axon, plasticity.
6. Why must an artificial neuron have an activation function?
7. Give three differences between biological and artificial neurons.
8. Which kind of model would you pick for: a 3,000-row sales table? 1 million labelled X-rays? Writing product descriptions?

Answers

1. AI ⊃ ML ⊃ DL ⊃ GenAI. ChatGPT is in all four. The chess program is AI only (no learning from data).
2. Data requirements, feature engineering (manual vs automatic), architecture.
3. A human (manual feature engineering). The network itself (automatic).
4. "AI = ML" (no: rule-based AI doesn't learn). "GenAI is a different technology" (no: it's scaled deep learning). Also "bigger is always better" (no: classical ML can win on small structured data).
5. Inputs, weights, weighted sum + bias, activation function, output, weight updates (backpropagation).
6. Without non-linearity, stacked layers collapse into one linear function and can't learn complex patterns. It also mirrors the neuron's firing threshold.
7. Any three of: electrochemical vs numerical signals; asynchronous vs synchronous timing; poorly understood biochemical learning vs exact backpropagation; huge vs simple

complexity; 20 W vs huge energy use; ~100 trillion synapses vs fewer parameters.

8. Classical ML. DL (a CNN). GenAI (an LLM).

03 · The perceptron (lecture of 06-08-2026)

Notebook: `notebooks/03_perceptron_from_scratch.ipynb` (both worked examples recomputed, a perceptron built from scratch, learning-rate and XOR experiments)

---

1. What is a perceptron?

> The **simplest unit of a neural network**: a single artificial neuron that takes numeric inputs, combines them with a **weighted sum**, applies an **activation function**, and outputs a **binary** (0/1) decision.

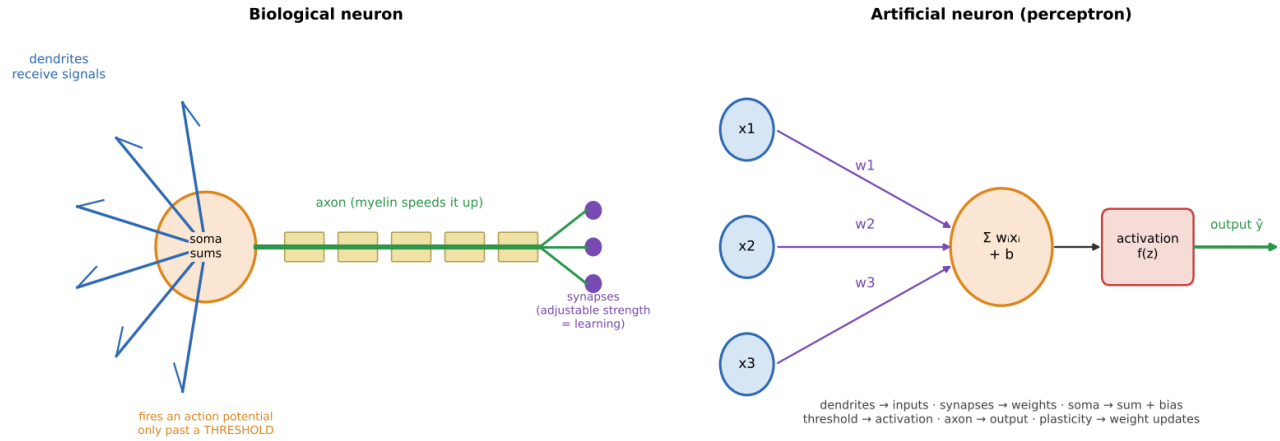
The lecture's running example: predict whether a student will **Pass (1) or Fail (0)** from **x1 = hours of study** and **x2 = hours of practice**.

Symbol	Meaning
x1, x2	inputs (study hours, practice hours)
w1, w2	weights: importance of each input
b	bias: shifts the threshold
z	net input (weighted sum): z = w1·x1 + w2·x2 + b
f(z)	activation function (the **step** function)
ŷ (y_hat)	predicted output (0 = Fail, 1 = Pass)
y	actual (true) answer
η (eta)	learning rate: size of each correction

2. Architecture

The lecture draws three conceptual layers: **input** (x1, x2) → the **neuron** (weighted sum, then activation) → **output** (ŷ). Inside the neuron there are two stages: ① compute z, ② apply f(z).

> Careful with words: the lecture labels the neuron's column "hidden layer". In standard terminology a single perceptron has **no** hidden layer: its neuron *is* the output unit. "Hidden layers" are what multi-layer perceptrons add (section 7). Use the lecture's wording in the lecture's exam, and know the standard meaning.



3. Forward propagation

Information flowing **left to right**: inputs → weighted sum → activation → prediction.

$z = \Sigma\, w_i x_i + b$, $\hat{y} = f(z)$

Worked example (lecture 3.2), recomputed in the notebook

Given x1 = 2, x2 = 3, w1 = 0.5, w2 = 0.8, b = −2: 1. Weighted products: x1·w1 = 2 × 0.5 = **1.0** and x2·w2 = 3 × 0.8 = **2.4**. 2. Net input: z = 1.0 + 2.4 − 2.0 = **1.4**. 3. Step activation: f(1.4) = **1**, since 1.4 ≥ 0. 4. **$\hat{y}$ = 1 → Pass.**

4. The activation: Heaviside step function

f(z) = 1 if z ≥ 0, f(z) = 0 if z < 0

It turns the continuous z into a crisp yes/no. Note that **z = 0 counts as 1** in the lecture's definition. (Chapter 04: the step function's slope is 0 everywhere, which is why modern networks use smooth activations.)

5. Learning: the perceptron update rule

Forward propagation only **predicts**. Learning compares $\hat{y}$ with the truth y and corrects the weights:

$w_i(\text{new}) = w_i(\text{old}) + \eta\cdot(y - \hat{y})\cdot x_i$ $b(\text{new}) = b(\text{old}) + \eta\cdot(y - \hat{y})$

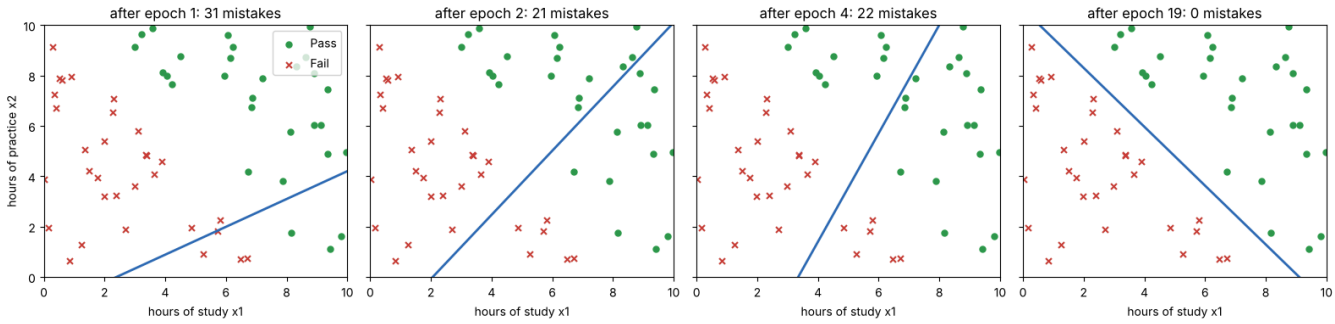
The error $(y - \hat{y})$ can only be:

$y - \hat{y}$	Meaning	Effect
0	correct	no change
+1	said Fail, was Pass (**under-predicted**)	weights move **up**, in proportion to each input
−1	said Pass, was Fail (**over-predicted**)	weights move **down**

Worked example (lecture 5.2), recomputed in the notebook

Given x1 = 2, x2 = 3, w1 = 0.2, w2 = −0.4, b = −1.0, y = 1, ŷ = 0, η = 0.1: 1. Error = 1 − 0 = **+1** (under-prediction). 2. w1 = 0.2 + 0.1·(1)·(2) = **0.4**. 3. w2 = −0.4 + 0.1·(1)·(3) = **−0.1**. 4. b = −1.0 + 0.1·(1) = **−0.9**.

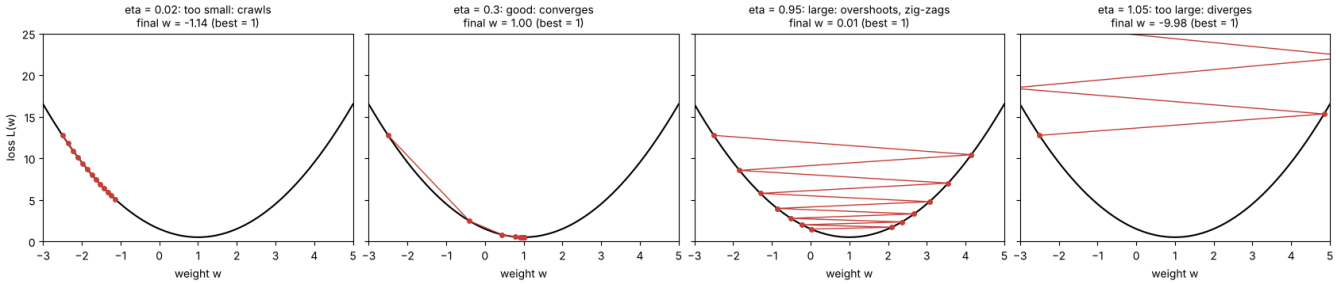
Notice (from the notebook): - Each weight moved **in proportion to its input** (w2 moved more, because x2 = 3 > x1 = 2). - After this one update z went from −1.8 to −0.4. That's **closer, but still wrong** (ŷ is still 0). It takes several passes over the data.



Notebook: the decision line moves as the perceptron learns on 60 students, reaching 0 mistakes after 19 epochs.

6. Gradient descent and the learning rate

Repeating updates over many examples is like **walking downhill on a loss landscape**: each step moves the weights in the direction that reduces the error, until reaching the minimum.



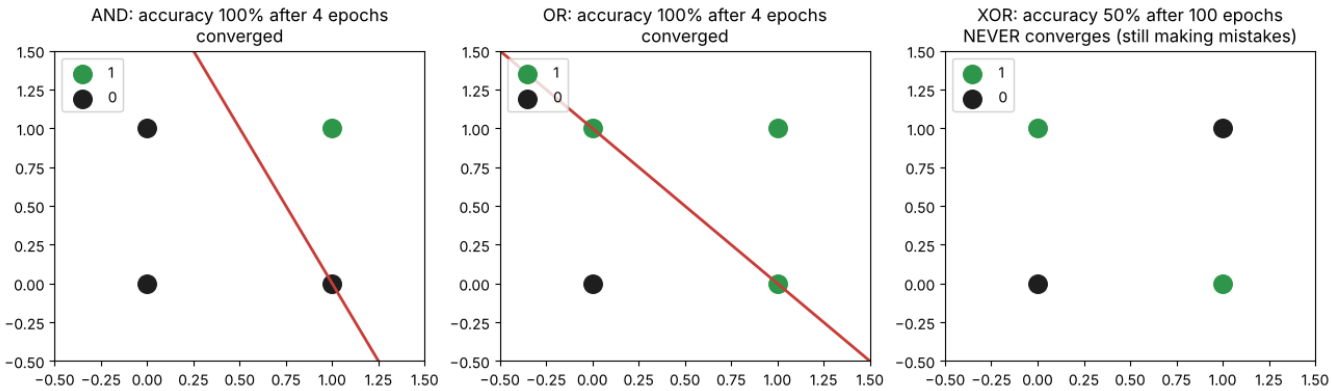
Learning rate	What happens
too small (η → 0)	**very slow**: thousands of tiny steps
good	converges quickly
too large (η ≫ 0)	**overshoots** the minimum, and the loss **bounces** or **diverges**
local vs global minimum	a well-tuned η helps reach the true (global) minimum instead of getting stuck in a shallow local one

> **A surprise the notebook found:** for the classic perceptron starting from **zero weights**, every η (0.001 to 10) took **exactly the same 19 epochs**. The step function only looks at the **sign** of z, and scaling all the weights and the bias by η doesn't change any sign. From non-zero starting weights, η did matter (3 to 19 epochs). The trade-off in the table is really about **gradient descent on a smooth loss**, which all modern networks use, and the notebook's loss-curve plot shows it exactly.

7. Linear separability: what one perceptron can and can't learn

A perceptron draws **one straight line** (w1·x1 + w2·x2 + b = 0). So it can only learn **linearly separable** functions:

x1	x2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

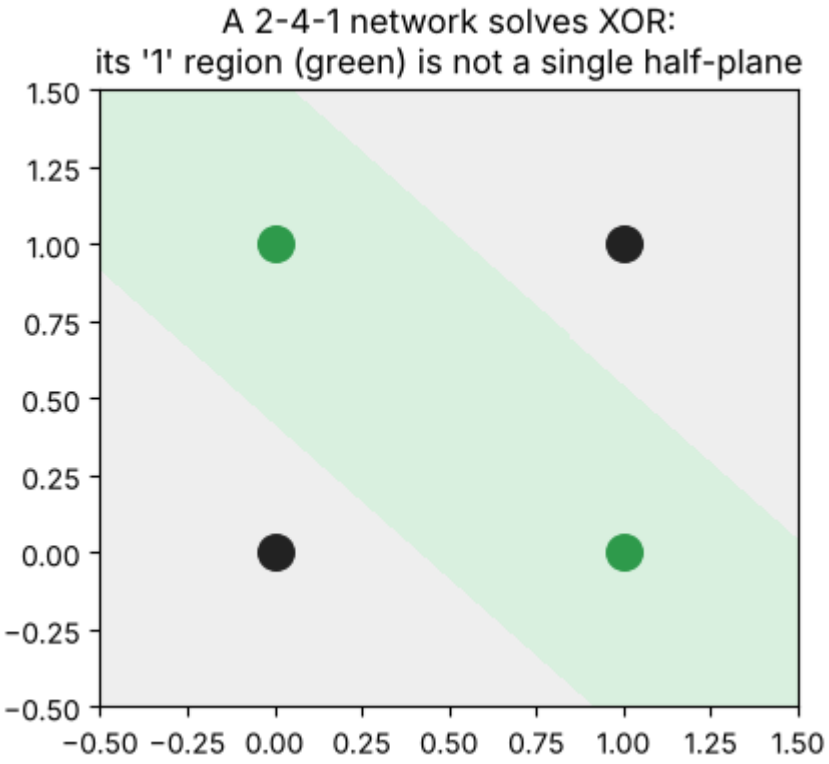


- **AND and OR:** one line separates the 1s from the 0s, and the perceptron learns them (4 epochs each in the notebook).
- **XOR:** the 1s sit on one diagonal and the 0s on the other, so **no straight line works**. The notebook tried **200,000 random lines**, and none beat **75%** (3 of 4 points). This is **Minsky & Papert's 1969** result, which helped start the AI winter (chapter 01).

  Zero margin, seen in the notebook: the learned AND line passes **exactly through the point (1, 0)**. z there is −0.000000000000000028, a floating-point rounding hair. The perceptron stops as soon as it makes no mistakes, however close a point sits to the line. Methods that **maximise the margin** (SVMs) exist to fix this.

The fix: hidden layers (multi-layer perceptron)

XOR = (x1 OR x2) AND NOT (x1 AND x2). Two hidden neurons (OR, NAND) and one output neuron (AND) solve it. The notebook does this with hand-set weights, then **learns** it with a 2-4-1 network trained by backpropagation (outputs 0.003, 0.997, 0.997, 0.003).



Stacking neurons lets the network **bend** its decision boundary into curves and regions, which is why deep networks exist. (Gradient training needs a **smooth** activation such as sigmoid instead of the step, see chapter 04.)

---

8. Summary (the lecture's revision box)

1. **Forward propagation:** $z = \Sigma\, w_i x_i + b$, then $\hat{y} = f(z)$.
2. **Step activation:** 1 if $z \geq 0$, else 0.
3. **Error:** $(y - \hat{y}) \in \{-1, 0, +1\}$.
4. **Update rule:** $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$ and $b \leftarrow b + \eta(y - \hat{y})$.
5. **Gradient descent:** repeated updates walk the weights downhill towards the minimum loss.
6. **Core limitation:** a single perceptron only solves **linearly separable** problems (AND, OR), **not XOR**.

Practice problems (with answers)

1. $x = (1, 4)$, $w = (0.5, -0.5)$, $b = 1$. Compute z and $\hat{y}$.
2. A perceptron predicts $\hat{y} = 1$ for $x = (2, 1)$ but $y = 0$, with $\eta = 0.5$, $w = (0.3, 0.6)$, $b = 0$. Give the new w and b.
3. With $w = (1, 1)$, what bias makes the perceptron compute AND? OR?
4. Why does the update rule never change the weights for a correctly classified example?
5. Explain in two sentences why XOR needs a hidden layer.
6. Your training loss bounces up and down and grows. What do you change?
7. Is the perceptron supervised or unsupervised? Why?

Answers

1. $z = 0.5 \cdot 1 + (-0.5) \cdot 4 + 1 = 0.5 - 2 + 1 = -0.5 \rightarrow \hat{y} = 0$.
2. Error $= 0 - 1 = -1$. $w_1 = 0.3 + 0.5 \cdot (-1) \cdot 2 = -0.7$. $w_2 = 0.6 + 0.5 \cdot (-1) \cdot 1 = 0.1$. $b = 0 + 0.5 \cdot (-1) = -0.5$.
3. AND: b between -2 and -1, e.g. $b = -1.5$ (only $1 + 1 = 2$ crosses zero). OR: b between -1 and 0, e.g. $b = -0.5$.
4. The error $(y - \hat{y})$ is 0, so the update $\eta \cdot 0 \cdot x$ is 0.
5. A single perceptron draws one straight line, and XOR's classes lie on opposite diagonals, so no line separates them. A hidden layer combines several lines (e.g. OR and NAND) into a non-linear region.
6. Lower the learning rate (the steps are overshooting the minimum).
7. Supervised: every training example comes with the correct label y, which the update rule uses.

04 · Activation functions (lectures of 07-08-2026 and 11-08-2026)

Part 1 (07-08): what an activation function is, why we need one, Step, Linear. **Part 2 (11-08):** Sigmoid, tanh, ReLU, Leaky ReLU, PReLU, vanishing gradients vs dying ReLU, which to use where.

Notebook: `notebooks/04_activation_functions.ipynb` (plots of every function and its slope, the linear-collapse proof, vanishing and dying gradients measured, an honest comparison)

---

1. What is an activation function?

> A mathematical function applied to a neuron's **weighted sum**, turning it into the output passed to the next layer.

$z = \Sigma\, w_i x_i + b \rightarrow \text{output} = f(z)$

Inputs → weighted sum + bias → activation function → output

2. Why we need one

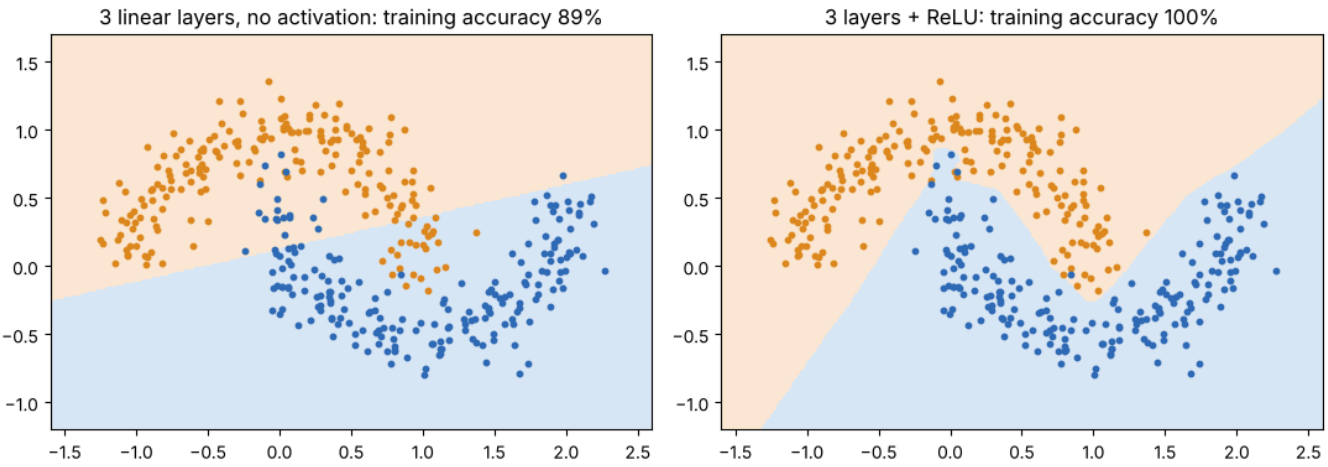
- **Non-linearity:** real problems are rarely linear, and non-linear activations let networks learn complex relationships.
- **Depth that means something:** stacking non-linear layers builds increasingly complex representations.
- **Useful decision boundaries:** curved, complex classification regions.
- **Controlled output ranges** (e.g. probabilities between 0 and 1).

Why linear layers alone are not enough

If every layer is linear, the whole network is **still one linear function**:

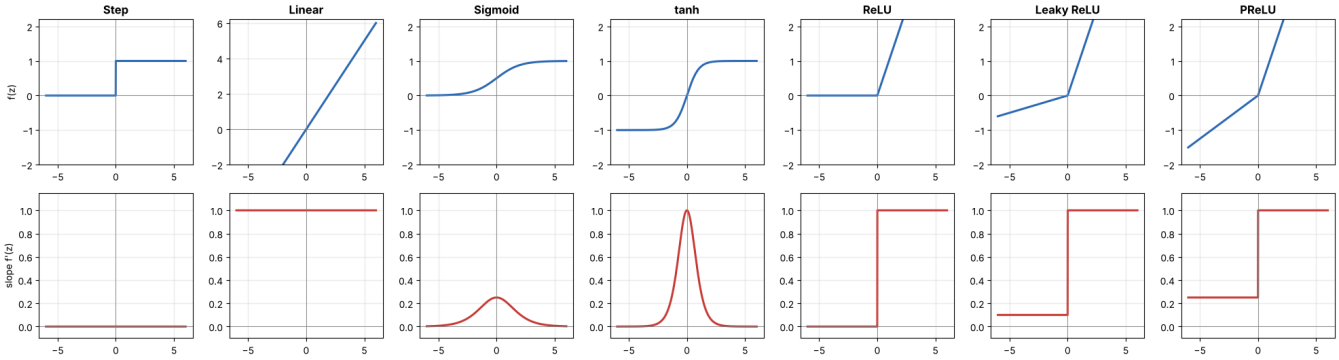
$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$

so adding more linear layers adds **no** power. The notebook **proves** it: a 4-layer linear network gives identical outputs to one collapsed layer, and a "deep" linear network on the two-moons data could only draw a straight line, while the same network with ReLU separated the moons.



---

3. The seven functions



Top row: the function. Bottom row: its slope (derivative), which is what backpropagation multiplies through the layers.

Step (Part 1)

- **f(z) = 1 if z ≥ 0, else 0.** A hard binary threshold (z = 2.5 → 1, z = −1.2 → 0).
- **Pros:** simple, a clear binary decision, historically important (the perceptron).
- **Cons: not differentiable at 0**, and its **derivative is 0 almost everywhere**, so it gives **no useful gradient** for backpropagation.
- **Use:** explaining the classic perceptron and hard binary decisions. **Not** used in modern deep networks.

Linear (Part 1)

- **f(z) = z.** Output proportional to input, unbounded.
- **Pros:** simple, cheap, easy to differentiate (slope 1).
- **Cons: no non-linearity**, so a network made only of linear layers stays linear. Unsuitable for hidden layers.
- **Use:** the **output layer for regression** (predicting an unbounded continuous number, e.g. a price).

	Step	Linear
output	discrete (0/1)	continuous

	Step	Linear
shape	threshold	straight line
differentiable	no (at 0)	yes
non-linear	yes	**no**
hidden layers	generally not used	generally not used
typical use	classical perceptron	regression output

Sigmoid (Part 2)

- **$\sigma(z) = 1 / (1 + e^{-z})$**, with derivative **$\sigma(z)\cdot(1 - \sigma(z))$**.
- Range **(0, 1)**, an S-shape. σ(0) = 0.5.
- Smooth, but **not zero-centred**. It **saturates** near 0 and 1, and its **maximum slope is only 0.25**.
- **Pros:** probability-like output, smooth, good for binary decisions and **gates** (LSTMs).
- **Cons: vanishing gradients** when saturated; not zero-centred; a poor default for deep hidden layers; the exponential is costlier than piecewise functions.
- **Use:** the **output layer of binary classifiers**.

tanh (Part 2)

- **$\tanh(z) = (e^{z} - e^{-z})/(e^{z} + e^{-z})$**, with derivative **$1 - \tanh^2(z)$**.
- Range **(−1, 1)**, **zero-centred** (tanh(0) = 0), the key difference from sigmoid. Largest slope (1) at 0, and it **saturates** near ±1.
- **Pros:** zero-centred outputs, bounded hidden states, historically important in RNNs.
- **Cons:** vanishing gradients when saturated, so large inputs slow learning. Often replaced by ReLU-family activations in deep feed-forward networks.
- **Use: RNN hidden states, LSTM/GRU state computations**, anywhere bounded zero-centred values are wanted.

ReLU, Rectified Linear Unit (Part 2)

- **f(z) = max(0, z).** Range **[0, ∞)**, piecewise linear. Slope **1** for z > 0 and **0** for z < 0.
- Negative inputs give exactly 0, so activations are **sparse**.
- **Pros: fast**; the strong default for deep hidden layers; **no saturation on the positive side**; sparsity can help.
- **Cons: dying ReLU** (a neuron stuck on the negative side stops learning); zero gradient for negatives; not zero-centred; unbounded positive output.
- **Use: hidden layers in CNNs and most feed-forward networks**.

Leaky ReLU (Part 2)

- **f(z) = z if z > 0, else α·z** (small fixed α, e.g. 0.01). Slope 1 or α.
- **Pros:** gradient still flows for negative inputs, **less prone to dying neurons**, simple and cheap.
- **Cons:** you must **choose α**, which may not suit every dataset; still not zero-centred.
- **Use:** hidden layers when ReLU leaves too many neurons inactive.

PReLU, Parametric ReLU (Part 2)

- **f(z) = z if z > 0, else a·z**, where **a is learned** during training (one per layer, or one per channel/unit).
- **Pros: learns the negative slope from the data**, reduces dying-neuron risk, more flexible than a fixed α.
- **Cons: extra trainable parameters**, can increase overfitting on small data, more complex, benefits depend on the architecture and data.
- **Use:** deep CNNs, and experiments where an adaptive slope may help.

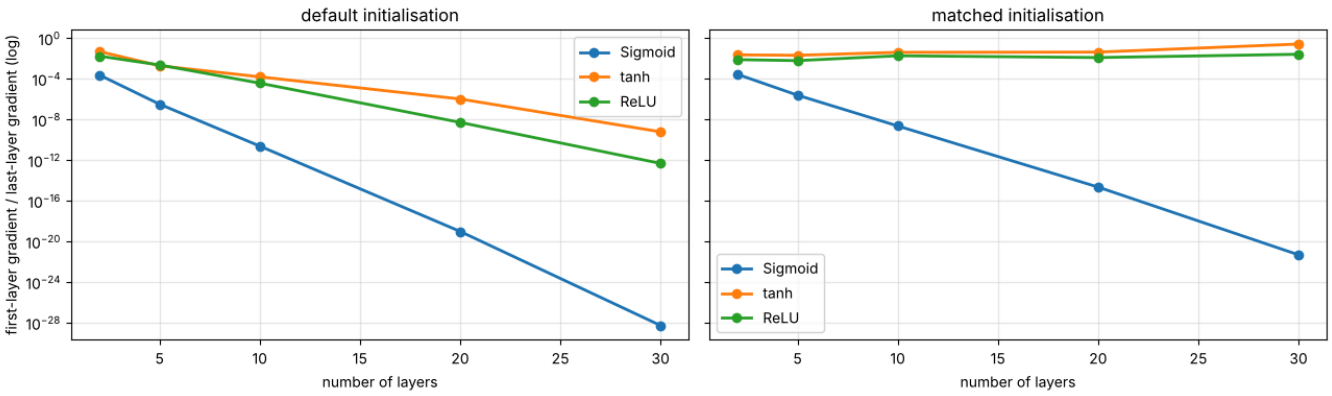
	ReLU	Leaky ReLU	PReLU
negative input →	0	α·z	a·z
negative-side gradient	0	fixed α	learned a
dying-neuron risk	higher	lower	lower
trainable slope	no	no	**yes**
core idea	rectify	leak a little	**learn** the leak

---

4. Vanishing gradient vs dying ReLU (don't mix them up)

	Vanishing gradient	Dying ReLU
cause	derivatives are **small** (sigmoid ≤ 0.25, saturated sigmoid/tanh ≈ 0) and get **multiplied** over many layers	the derivative is **exactly 0** for negative inputs
who suffers	sigmoid, tanh (in saturation)	ReLU
effect	**early layers** learn extremely slowly	a neuron whose input stays negative **never updates again**
remedies	ReLU-family activations, good initialisation, BatchNorm	Leaky ReLU, PReLU (non-zero negative slope), sensible learning rates

What the notebook measured



- **Vanishing gradient.** In a 30-layer **sigmoid** network the gradient reaching layer 1 was about **10^{-22}** of the last layer's, even with the best initialisation, so those layers can't learn. **tanh and ReLU kept a steady gradient only with matching initialisation**: Xavier for tanh, **He** for ReLU. With PyTorch's default starting weights, they also lost most

of the signal. So "ReLU fixes vanishing gradients" really means "ReLU **with He initialisation**".

- **Dying ReLU, one neuron.** With its bias pushed to −2, a ReLU neuron's gradient was **exactly 0** for 2,000 steps (dead forever). The Leaky ReLU neuron recovered and fitted the data.
- **Dying ReLU, a whole network** with 77% of its neurons knocked out (biases −0.35):
- ReLU stayed at chance (10%).
- Leaky ReLU with α = 0.01 *and* with α = 0.1 **also** stayed stuck in 60 epochs: the small leak through three layers was too weak for plain SGD.
- **PReLU learned its slope and recovered to 97.8%.**
- Leaky ReLU **reduces** the risk; it doesn't remove it.
- **PReLU's learned slopes** ended at 1.79, 1.34 and 2.06, from a start of 0.25. Nothing forces them to stay small.
- **Head-to-head** (6 hidden layers on digits, Adam, 3 seeds):
- sigmoid **27%**, tanh 95.2%, ReLU 91.0%, Leaky ReLU 93.1%, PReLU 94.6%, all with default initialisation;
- **ReLU + He initialisation 96.9% (best)**.

---

5. Which activation where (the lecture's table)

Situation	Recommended	Reason
binary classification **output**	**Sigmoid**	output between 0 and 1
regression **output**	**Linear**	unbounded continuous value
typical **hidden layers**	**ReLU**	fast, strong baseline (use He initialisation)
dying-ReLU concern	**Leaky ReLU**	non-zero negative gradient
adaptive negative slope wanted	**PReLU**	slope is learned
bounded, zero-centred hidden state	**tanh**	range −1 to +1
explaining the classic perceptron	**Step**	hard 0/1 threshold

Function	Range	Zero-centred	Strength	Limitation	Typical use
Sigmoid	(0, 1)	no	probability-like output	saturation	binary output
tanh	(−1, 1)	**yes**	zero-centred	saturation	RNNs / bounded states
ReLU	[0, ∞)	no	fast and sparse	dying neurons	hidden layers
Leaky ReLU	(−∞, ∞)	no	negative gradient	fixed α	hidden layers
PReLU	(−∞, ∞)	no	learned negative slope	extra parameter	CNNs

Summary (lecture): - Sigmoid is mainly an **output** activation for binary predictions. - tanh is zero-centred, good for bounded hidden states, but saturates. - ReLU is the strong default for hidden layers and gives sparse activations. - Leaky ReLU keeps a small negative gradient against dying ReLU. - PReLU learns the negative slope, at the cost of extra parameters. -

Choose by layer, task and optimisation behaviour, and validate empirically. The notebook's results show why that last point matters.

Check yourself

1. Compute σ(0), tanh(0), ReLU(−3), LeakyReLU(−3) with α = 0.01.
2. What is the maximum derivative of sigmoid? Of tanh? Where do they occur?
3. Why can't you train a network of step functions with backpropagation?
4. Why is a network of only linear layers no better than one layer?
5. Explain the difference between the vanishing-gradient problem and dying ReLU.
6. Leaky ReLU vs PReLU: what is fixed and what is learned?
7. Choose activations for: hidden layers of a CNN; the output of a spam detector; the output of a temperature predictor; an LSTM's hidden state.
8. Which activation is zero-centred, sigmoid or tanh? Why does that matter?

Answers

1. 0.5, 0, 0, −0.03.
2. Sigmoid: 0.25 at z = 0. tanh: 1 at z = 0.
3. Its derivative is 0 everywhere except the jump, where it's undefined, so there's no gradient signal.
4. A composition of linear functions is linear: $W_2(W_1x + b_1) + b_2 = Wx + b$.
5. Vanishing: slopes are small but non-zero (saturating sigmoid/tanh) and shrink when multiplied across layers. Dying ReLU: the slope is exactly 0 because the neuron's input is always negative, so it never updates.
6. Leaky ReLU: the slope α is a fixed hyperparameter. PReLU: the slope a is a trainable parameter.
7. ReLU (Leaky ReLU or PReLU if neurons die); sigmoid; linear; tanh.
8. tanh. Zero-centred outputs keep the next layer's inputs balanced around 0, which helps optimisation (sigmoid outputs are always positive).

05 · Normalization and regularization (lecture of 04-09-2026)

Notebook: `notebooks/05_normalization_and_regularization.ipynb` (scaling vs gradient descent, leakage, BatchNorm and LayerNorm by hand, every regulariser tested on an overfitting network)

---

1. Two different problems

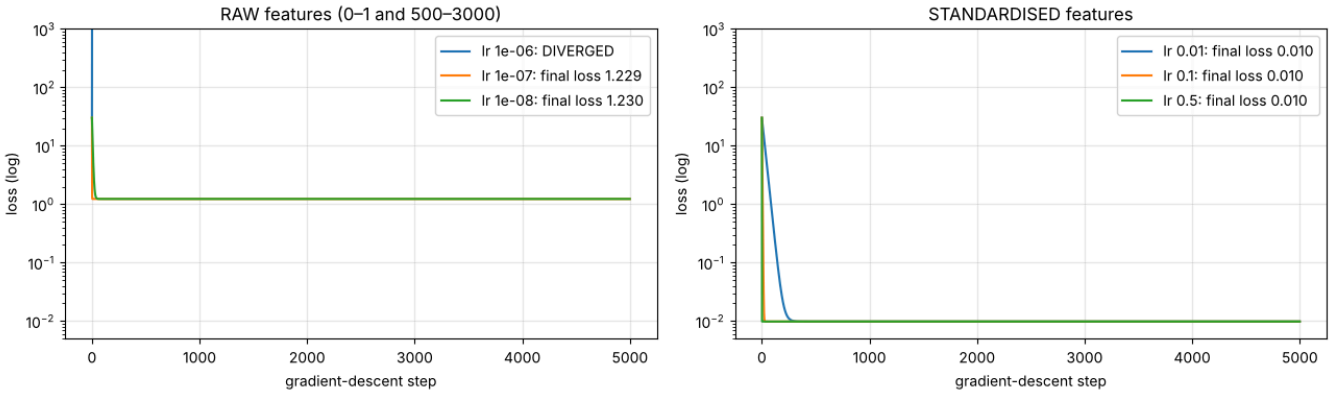
Concept	Main purpose	Examples
Normalization	control the **scale/distribution** of data and activations → **stable, efficient optimisation**	min-max scaling, standardisation, BatchNorm, LayerNorm
Regularization	control **model complexity**, reduce **overfitting** → **better generalisation** to unseen data	L1, L2, dropout, early stopping, augmentation, weight decay

They solve different problems and are **complementary**: a model can be normalised **and** regularised at the same time.

Part A · Normalization

2. Why normalisation is needed

If one feature lies in 0–1 and another in the millions, their gradients are on very different scales, and gradient descent struggles. Scaling makes the problem easy to navigate.



Notebook result: predicting a price from a 0–1 rating and a 500–3,000 floor area. - **Raw features:** learning rate 10^{-6} **diverged**; 10^{-7} and 10^{-8} **stalled** at loss 1.23 after 5,000 steps (the small feature barely learns at such tiny rates). - **Standardised features:** the same model reached the noise floor (loss 0.0098) in 199, 19, or even 1 step(s) at learning rates 0.01, 0.1, 0.5.

2.1 Standardisation (z-score)

$x' = (x - \mu) / \sigma$, with μ and σ from the **training** data. - centres the feature at 0, with standard deviation ≈ 1 on the training set; - a strong default for continuous numerical features.

2.2 Min-max scaling

$x' = (x - \text{min}) / (\text{max} - \text{min})$ → about [0, 1]. - Useful when a bounded range is wanted. - **Sensitive to outliers**, because the extremes set the range. Notebook: with one 1,000k salary among 99 normal ones, min-max squashed all 99 normal values into **5%** of the range.

2.3 Image normalisation

8-bit pixels **0–255 → 0–1** (divide by 255), so the optimiser works with manageable numbers.

2.4 Avoid data leakage

> Compute μ, σ (or min/max) from the **training set only**, then apply **those same values** to validation and test data.

Never compute normalisation statistics on the test set and then report test performance as if it were unseen. Information has leaked, and the score is optimistic.

3. Normalization layers (inside the network)

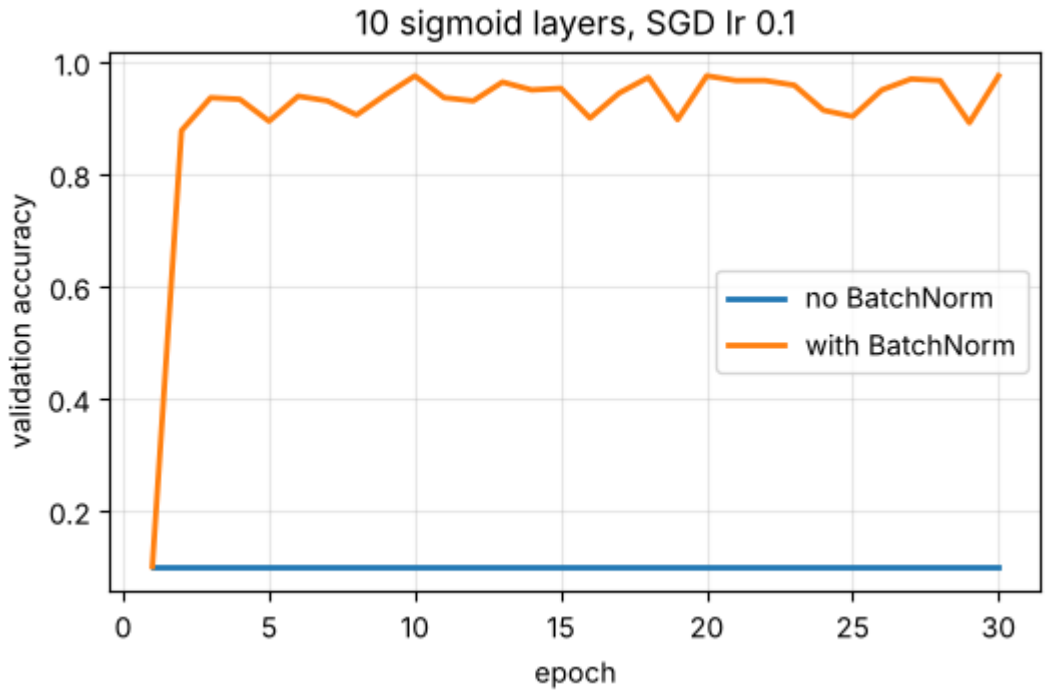
Method	Main idea	Common use
Batch Normalization	mini-batch statistics for each feature/channel	CNNs, many feed-forward networks
Layer Normalization	normalises the features **within each sample**	Transformers, sequence models
Instance Normalization	spatial features per sample/channel	style transfer, image generation
Group Normalization	groups of channels	CNNs with **small batches**

4. Batch Normalization

For each feature, over a mini-batch of size m: 1. **mean** $\mu_B = (1/m)\ \Sigma\ x_i$ 2. **variance** $\sigma^2_B = (1/m)\ \Sigma\ (x_i - \mu_B)^2$ 3. **normalise** $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$ 4. **scale and shift** $y_i = \gamma \cdot \hat{x}_i + \beta$, where **γ and β are learned**

Key points: - **Training:** uses **batch statistics**. - **Inference:** uses **moving (running) averages** collected during training. - **ε** prevents division by zero or by a tiny number. - **γ, β** keep the layer able to learn whatever scale and offset is useful (it can even undo the normalisation).

Notebook: - The hand computation matched `nn.BatchNorm1d` exactly. - In train mode, a "batch" of identical rows normalised to all zeros (variance 0). In eval mode the running statistics were used instead, which is why `model.eval()` matters. - A 10-layer **sigmoid** network learned **nothing** in 30 epochs without BatchNorm (10% = chance), but reached **89% after 5 epochs** and **97.5% after 30** with BatchNorm after every layer.



5. Layer Normalization

The same formulas, but the mean and variance are computed **across the H features of one sample**, then γ and β are applied.

BatchNorm	LayerNorm
relies on **batch** statistics during training	works **independently for each sample**
can be hurt by **very small** batches	handles small or variable batch sizes naturally
behaves differently in train and eval	identical in train and eval
common in **CNNs**	common in **Transformers**

Part B · Regularization

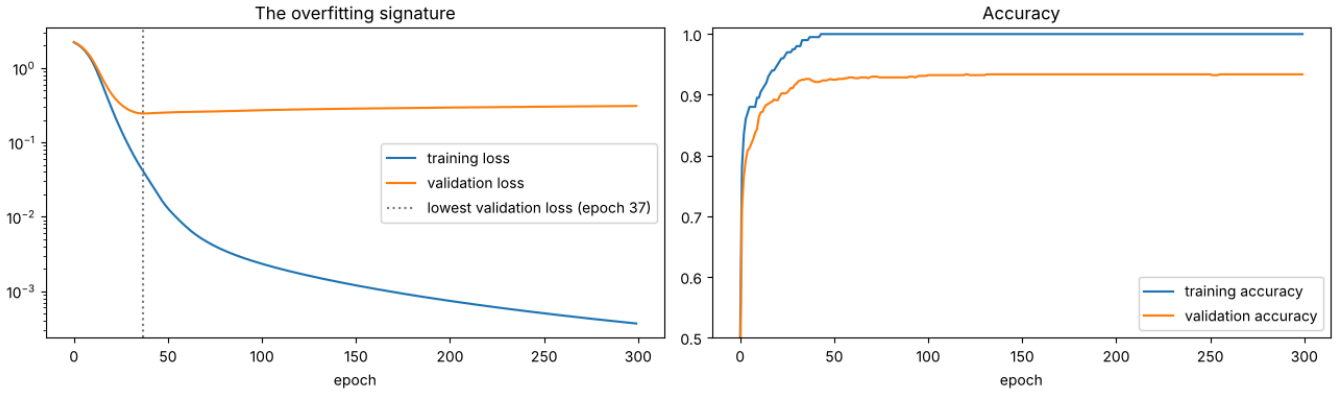
6. What is regularization?

> Methods that stop a network from **fitting noise or memorising** the training set, so it performs better on **unseen** data.

Condition	Training	Validation/Test
underfitting	poor	poor
good generalisation	good	good
overfitting	**very good**	**worsening / poor**

7. How to detect overfitting

> **Training loss keeps falling while validation loss stops improving or starts rising.**



Notebook: a 300,000-weight network on just 200 digit images. - **Training:** 100% accuracy (memorised). - **Validation loss:** bottomed out at 0.245 around epoch 37, then rose to 0.309 by epoch 300. - **Validation accuracy** just plateaued (93.4%), so **compare losses, not just accuracy**. Establish a clear **baseline** before adding regularisation.

8. L1 regularization

Loss = original loss + λ Σ $|w_i|$. The (sub)gradient adds λ·sign(w), a constant push towards zero. - Main effect: **sparsity** (many weights become exactly or nearly zero), so it's useful when you want feature selection. - Too large a λ removes too much capacity → **underfitting**.

9. L2 regularization

Loss = original loss + (λ/2) Σ w_i^2. The gradient adds **λ·w**, so larger weights shrink more. - Shrinks weights **smoothly** without forcing many to exactly zero. - The general-purpose default.

	L1	L2
penalty	$\|w\|$	$w^2/2$

	L1	L2
main effect	**sparsity**	**weight shrinkage**
exactly-zero weights	**more likely**	less likely
smooth at zero	no	yes
typical purpose	sparse models, feature selection	general weight control

Notebook: L1 ($\lambda = 10^{-4}$) left **88%** of the weights near zero, against 24% for L2 and 2% with no regularisation.

10. Elastic Net

Loss + $\lambda_1 \Sigma |w| + (\lambda_2/2) \Sigma w^2$: a compromise between L1's sparsity and L2's smooth shrinkage.

11. Dropout

Randomly **disable a fraction p** of activations on each training step. - The network can't depend on particular neurons, so it learns **distributed** representations. - **Inverted dropout:** the survivors are scaled by $1/(1 - p)$ during training, so nothing needs changing at test time. - **Disabled during inference** (`model.eval()`). - It is **stochastic** (a different mask each update). - **Too much dropout → underfitting.**

12. Early stopping

Stop training when **validation** performance stops improving, and keep the parameters from the **best validation epoch**. - **Patience** allows temporary ups and downs before stopping. - Use the **validation** set to decide, **never** the test set repeatedly.

Notebook: with patience 20, training stopped around epochs 56–70 instead of 300 (about 5× cheaper), with a similar test accuracy (93.2% vs 93.5%).

13. Data augmentation

Create **realistic variations** of the training examples: small rotations, crops, translations, flips, brightness changes. **The transformation must keep the label correct.** - More effective diversity, less overfitting, especially useful for small datasets. - **Wrong transformations create mislabelled or unrealistic examples.** Flipping a "6" can make a "9", for instance. In the notebook, 1-pixel shifts on tiny 8×8 digits (12% of the width, with wrap-around) **hurt** test accuracy (91.8% vs 93.5%): a live example of this warning.

14. Weight decay

Shrink the parameters directly at each step. - **SGD:** $w \leftarrow w - \eta(\nabla L + \lambda w) = (1 - \eta\lambda)\cdot w - \eta\nabla L$. That is "shrink, then step", which for plain SGD is equivalent to L2. - **Adam:** adding L2 to the loss is *not* the same as true weight decay, because Adam rescales each gradient. **Decoupled** weight decay applies the shrink directly, and that is the motivation for **AdamW**.

Notebook: - **Adam with** `weight_decay = 0.01` behaved like the L2 penalty: 24% of weights near zero. - **AdamW with the same 0.01** barely changed anything: each step only shrinks w by lr × wd = 0.001%. **Same number, different mechanism.** AdamW is normally used with larger values (e.g. 0.05–0.1).

All regularisers on one problem (notebook, 3 seeds, test accuracy)

Setting	Test accuracy	Weights ≈ 0
none	93.5%	2%
L2 ($\lambda = 10^{-3}$)	**94.1%**	24%
L1 ($\lambda = 10^{-4}$)	93.7%	**88%**
dropout p = 0.5	**93.9%**	2%
augmentation (1-px shift)	91.8%	2%
Adam weight_decay 0.01	93.7%	24%
AdamW weight_decay 0.01	93.5%	2%
early stopping	93.2%	—

Honest reading: with only 200 training images, gains are about half a point, the size of the seed noise. Regularisation **helps a little**; it's not magic. **More data** is the strongest cure for overfitting.

---

15. Normalization vs regularization

Question	Normalization	Regularization
primary objective	stable scale / optimisation	better generalisation
examples	standardisation, BatchNorm, LayerNorm	L1, L2, dropout, early stopping, augmentation, weight decay
prevents overfitting?	**not necessarily**	**yes, by design**
can be combined?	yes	yes

16. Practical selection guide (lecture)

Problem	Starting point	Reason
continuous features on very different scales	**standardisation**	comparable feature scales
image pixels 0–255	rescale to 0–1 (or the model's preprocessing)	manageable values
CNN with ordinary batch sizes	**BatchNorm**	effective activation normalisation
Transformer / sequence model	**LayerNorm**	independent of batch statistics

Problem	Starting point	Reason
overfitting in a dense network	**L2, dropout, early stopping**	control model complexity
need sparse weights	**L1**	many small or zero weights

Check yourself

1. Normalization vs regularization: one sentence each on what problem it solves.
2. Standardise x = 70 if the training mean is 50 and std is 10. Min-max scale x = 70 if min = 20, max = 120.
3. Why is min-max scaling risky with outliers?
4. What is data leakage in normalisation, and how do you avoid it?
5. Write the four BatchNorm steps. What are γ and β for? What changes at inference?
6. BatchNorm vs LayerNorm: what does each average over, and where is each used?
7. How do you recognise overfitting from training and validation curves?
8. L1 vs L2: penalty, effect on weights, typical use.
9. What does dropout do during training, and during inference?
10. Why does AdamW exist?

Answers

1. Normalization makes optimisation stable and efficient by controlling scale. Regularization reduces overfitting so the model generalises.
2. $(70 - 50)/10 = 2.0$. $(70 - 20)/(120 - 20) = 0.5$.
3. The extreme value sets the range, squashing all the normal values into a tiny interval.
4. Using statistics computed with test (or validation) data. Fit the scaler on training data only, then apply it to the others.
5. Batch mean → batch variance → normalise with ε → scale by γ and shift by β (learned, so the layer can choose its own scale and offset). At inference the running averages replace the batch statistics.
6. BatchNorm averages over the batch (per feature/channel) and is used in CNNs. LayerNorm averages over the features (per sample) and is used in Transformers.
7. Training loss keeps falling while validation loss stops improving or rises.
8. L1: $\lambda\Sigma|w|$, drives weights to zero (sparsity), used for feature selection. L2: $(\lambda/2)\Sigma w^2$, shrinks weights smoothly, the general default.
9. Training: randomly zeroes a fraction p of activations (scaling the rest by $1/(1 - p)$). Inference: off, so all neurons are used.
10. With adaptive optimisers like Adam, L2-in-the-loss is not true weight decay. AdamW decouples the decay and applies it directly to the weights.

01 · AI vs ML vs DL vs GenAI, in code

Lecture: *AI vs ML vs DL vs GenAI & the Biological Neuron Analogy* (05-08-2026), Part A · **Notes:** `../notes/02_ai_ml_dl_genai_and_the_neuron.md`

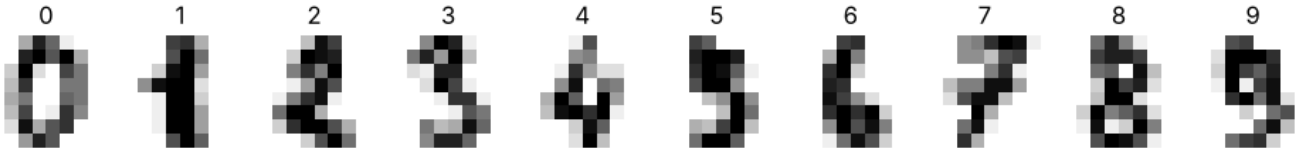
The lecture's hierarchy: **AI ⊃ ML ⊃ DL ⊃ GenAI**. The best way to feel the difference is to attack **one problem** at each level and compare:

Level	What we'll build	Who decides the features?
AI (rule-based, no learning)	hand-written if-then rules	a human writes the rules
ML (classical)	logistic regression on **hand-engineered** features	a human designs the features; the model learns weights
DL	a neural network on **raw pixels**	the network learns its own features
GenAI	a small generative network that **creates new digits**	learns the data distribution itself

The problem: scikit-learn's handwritten digits (1,797 images of 8×8 pixels, values 0–16). It ships with scikit-learn, so nothing to download. Everything runs in about a minute.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
digits = load_digits()
X, y, images = digits.data, digits.target, digits.images
X_tr, X_te, y_tr, y_te, img_tr, img_te = train_test_split(X, y, images, test_size=0.3,
random_state=0, stratify=y)
print("training images:", len(X_tr), " test images:", len(X_te))
fig, axes = plt.subplots(1, 10, figsize=(12, 1.6))
for d, ax in enumerate(axes):
    ax.imshow(images[y == d][0], cmap="gray_r"); ax.set_title(str(d)); ax.axis("off")
plt.show()
```

```
training images: 1257  test images: 540
```



Level 1 · AI without learning: hand-written rules

The 1960s–80s "expert system" way: a human looks at the data and writes rules. To keep it fair, we only try to tell **0 from 1** (the easiest pair). A plausible human rule: "a **1** is a thin vertical stroke, so its ink sits in the middle columns; a **0** is a ring, so it has ink at the left and right edges and a hole in the middle."

```
def rule_based_0_or_1(img):
    middle_ink = img[:, 3:5].sum()                # ink in the two centre columns
    side_ink = img[:, [1, 2, 5, 6]].sum()         # ink in the side columns
    centre_hole = img[3:5, 3:5].sum()             # a 0 has an empty centre
    return 1 if (middle_ink > side_ink * 0.5 and centre_hole > 10) else 0

mask = np.isin(y_te, [0, 1])
pred = np.array([rule_based_0_or_1(im) for im in img_te[mask]])
print(f"hand-written rules, 0 vs 1: {np.mean(pred == y_te[mask]):.1%} correct on {mask.sum()}
test images")
print("and for all 10 digits? We'd need rules for every digit and every handwriting style. Nobody
can write those by hand.")
```

```
hand-written rules, 0 vs 1: 97.2% correct on 109 test images
and for all 10 digits? We'd need rules for every digit and every handwriting style. Nobody can wr
ite those by hand.
```

That's **AI**: intelligent-looking behaviour from **rules a person wrote**, with no learning from data. It works for a narrow, clean task, but writing rules for all ten digits in every handwriting style is hopeless. That's the motivation for learning from data.

Level 2 · Machine learning with hand-engineered features

Classical ML: a **human chooses the features** (measurements), and the algorithm **learns** how to weigh them. We design 8 features a person might think of, then let logistic regression learn the weights.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

def engineered_features(img):
    ink = img > 4
    rows, cols = np.where(ink)
    return [img.sum(),                                    # total ink
            img[:4].sum() - img[4:].sum(),                # top-heavy or bottom-heavy?
            img[:, :4].sum() - img[:, 4:].sum(),          # left-heavy or right-heavy?
            img[3:5, 3:5].sum(),                          # ink in the centre (hole?)
            (cols.max() - cols.min()) if len(cols) else 0, # width
            (rows.max() - rows.min()) if len(rows) else 0, # height
            np.abs(np.diff(img, axis=0)).sum(),           # vertical edges
            np.abs(np.diff(img, axis=1)).sum()]           # horizontal edges

F_tr = np.array([engineered_features(im) for im in img_tr]); F_te =
np.array([engineered_features(im) for im in img_te])
ml = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(F_tr, y_tr)
print(f"ML on 8 hand-engineered features, all 10 digits: {ml.score(F_te, y_te):.1%}")
```

```
ML on 8 hand-engineered features, all 10 digits: 74.3%
```

It **learned** from data and handles all 10 digits, which is much better than rules. But it's limited by the **features a human thought of**. Ask yourself: what would be a good feature to tell 3 from 8, or 4 from 9? That's the human bottleneck the lecture calls **manual feature engineering**.

What if we give the same ML algorithm the **raw pixels** (64 numbers) instead?

```
ml_raw = make_pipeline(StandardScaler(), LogisticRegression(max_iter=5000)).fit(X_tr, y_tr)
print(f"the same logistic regression on 64 raw pixels: {ml_raw.score(X_te, y_te):.1%}")
```

```
the same logistic regression on 64 raw pixels: 97.2%
```

On tiny, centred, clean 8×8 digits, raw pixels are already a good representation, so the linear model does well. On real photos (millions of pixels, objects anywhere, varying lighting), raw pixels fed to a linear model work badly, and that is exactly where deep learning's **learned features** make the difference.

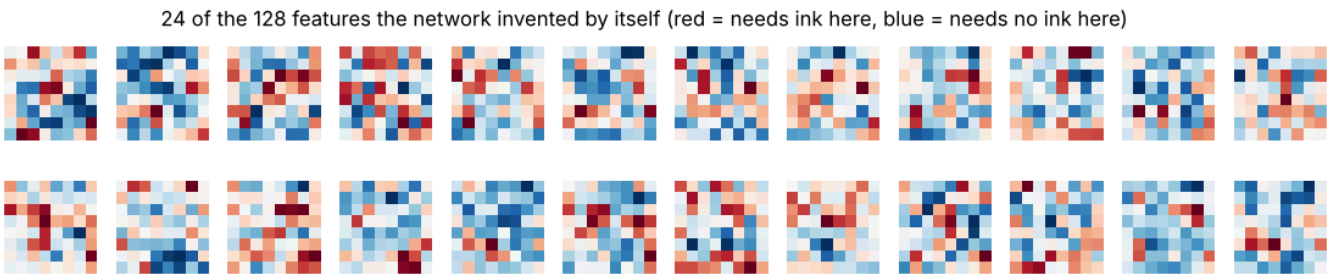
Level 3 · Deep learning: learn the features from raw pixels

A neural network with hidden layers takes the **raw pixels** and learns its own internal features in the hidden layers.

```
torch.manual_seed(0)
Xt_tr, Xt_te = torch.tensor(X_tr / 16, dtype=torch.float32), torch.tensor(X_te / 16,
dtype=torch.float32)
yt_tr, yt_te = torch.tensor(y_tr), torch.tensor(y_te)
dl = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(), nn.Linear(64,
10))
opt = torch.optim.Adam(dl.parameters(), lr=1e-3)
for epoch in range(60):
    perm = torch.randperm(len(Xt_tr))
    for i in range(0, len(perm), 64):
        idx = perm[i:i + 64]
        loss = nn.functional.cross_entropy(dl(Xt_tr[idx]), yt_tr[idx]); opt.zero_grad();
loss.backward(); opt.step()
with torch.no_grad():
    dl_acc = (dl(Xt_te).argmax(1) == yt_te).float().mean().item()
print(f"deep network (2 hidden layers) on raw pixels: {dl_acc:.1%}")

# What did the first hidden layer learn? Each neuron's 64 weights, drawn as an 8x8 picture
W1 = dl[0].weight.detach().numpy()
fig, axes = plt.subplots(2, 12, figsize=(14, 2.6))
for ax, w in zip(axes.ravel(), W1[:24]):
    ax.imshow(w.reshape(8, 8), cmap="RdBu"); ax.axis("off")
fig.suptitle("24 of the 128 features the network invented by itself (red = needs ink here, blue =
needs no ink here)")
plt.show()
```

```
deep network (2 hidden layers) on raw pixels: 97.6%
```



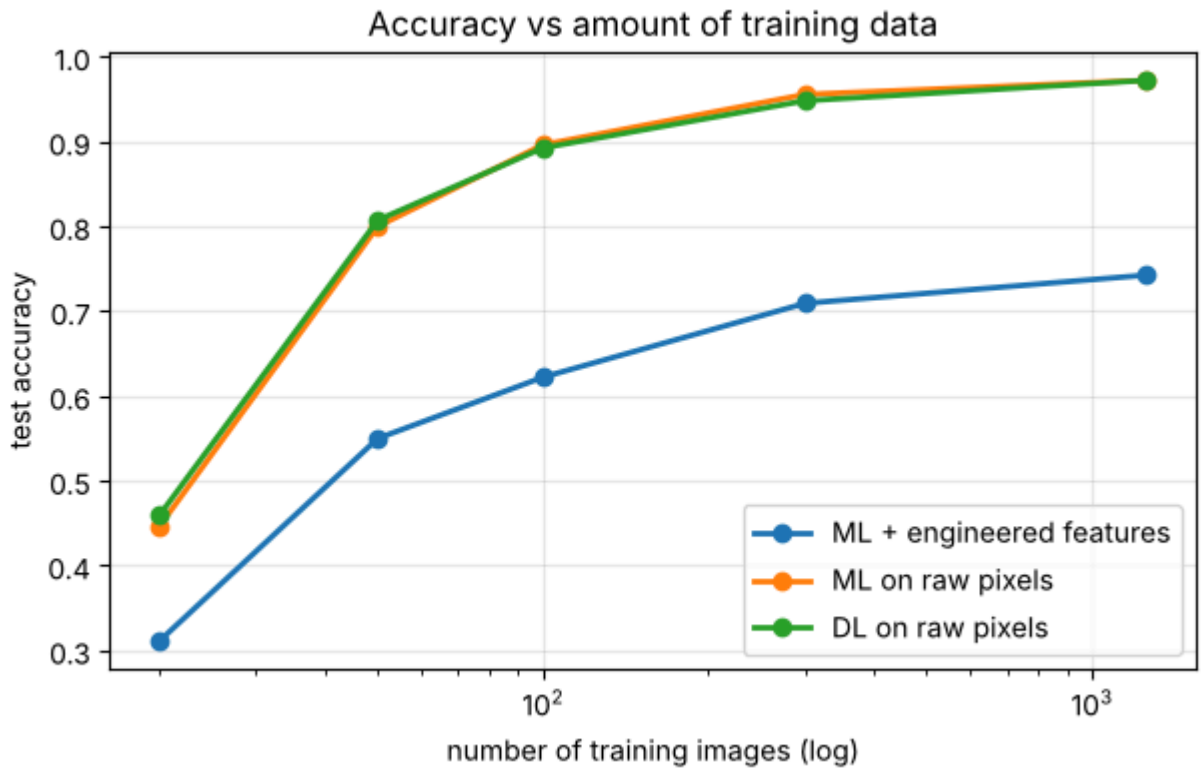
Nobody told the network to look for loops, strokes or centre holes. Its first layer **invented** its own detectors (the blobs and stroke patterns above). Deeper layers combine them. That's the lecture's **automatic, hierarchical feature learning**.

"DL needs more data": measured

The lecture says deep learning needs more data than classical ML. Let's check how accuracy grows with training-set size, for the three learners.

```python
def train_dl(Xa, ya, epochs=150, seed=0):
    torch.manual_seed(seed)
    net = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 64), nn.ReLU(),
nn.Linear(64, 10))
    o = torch.optim.Adam(net.parameters(), lr=1e-3)
    Xa, ya = torch.tensor(Xa / 16, dtype=torch.float32), torch.tensor(ya)
    for _ in range(epochs):
        perm = torch.randperm(len(Xa))
        for i in range(0, len(perm), 64):
            idx = perm[i:i + 64]
            l = nn.functional.cross_entropy(net(Xa[idx]), ya[idx]); o.zero_grad(); l.backward();
o.step()
    with torch.no_grad():
        return (net(Xt_te).argmax(1) == yt_te).float().mean().item()

sizes = [20, 50, 100, 300, len(X_tr)]
res = {"ML + engineered features": [], "ML on raw pixels": [], "DL on raw pixels": []}
for n in sizes:
    idx = np.random.default_rng(0).choice(len(X_tr), n, replace=False) if n < len(X_tr) else
np.arange(len(X_tr))
    res["ML + engineered features"].append(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=2000)).fit(F_tr[idx], y_tr[idx]).score(F_te, y_te))
    res["ML on raw pixels"].append(make_pipeline(StandardScaler(),
LogisticRegression(max_iter=5000)).fit(X_tr[idx], y_tr[idx]).score(X_te, y_te))
    res["DL on raw pixels"].append(train_dl(X_tr[idx], y_tr[idx]))
plt.figure(figsize=(7, 4))
for k, v in res.items():
    plt.plot(sizes, v, "o-", lw=2, label=k)
plt.xscale("log"); plt.xlabel("number of training images (log)"); plt.ylabel("test accuracy");
plt.legend(); plt.grid(alpha=0.3)
plt.title("Accuracy vs amount of training data"); plt.show()
print("train size:", sizes)
for k, v in res.items():
    print(f"{k:>25}: " + "  ".join(f"{a:.0%}" for a in v))
```




```
train size: [20, 50, 100, 300, 1257]
 ML + engineered features: 31%  55%  62%  71%  74%
         ML on raw pixels: 45%  80%  90%  96%  97%
         DL on raw pixels: 46%  81%  89%  95%  97%
```

How to read it (honestly):

- **The engineered-feature model is worst at every data size** (31% → 74%). My 8 hand-made features throw information away, and more data can't bring it back. That is the **human bottleneck** of manual feature engineering in action: the model can only be as good as the features someone thought of.
- **Logistic regression and the deep network on raw pixels track each other almost exactly** at every size (45% → 97% vs 46% → 97%). On tiny, clean, centred 8×8 digits, raw pixels are already an excellent representation, so the deep network's learned features have nothing extra to offer.
- So this dataset does **not** show "DL needs more data and then wins". That effect appears on **large, complex** data (real photos, speech, text), where a linear model on raw input is hopeless and only learned, layered features work, and where deep networks keep improving as data grows into the millions.
- It does confirm the lecture's misconception #4: **bigger is not always better**. On small, simple data, a classical model is just as accurate and far simpler.

Level 4 · Generative AI: create digits that never existed

Everything so far was **discriminative**: image in, label out. A **generative** model learns what the data itself looks like, so it can **produce new samples**.

We train a tiny **variational autoencoder (VAE)**: it squeezes each digit into 2 numbers (a "code"), learns to rebuild the digit from the code, and learns codes that are spread like a simple bell curve. Afterwards we pick **brand-new random codes** and the decoder **draws new digits**. (You don't need the VAE maths yet. The point is the generate-new-content behaviour.)

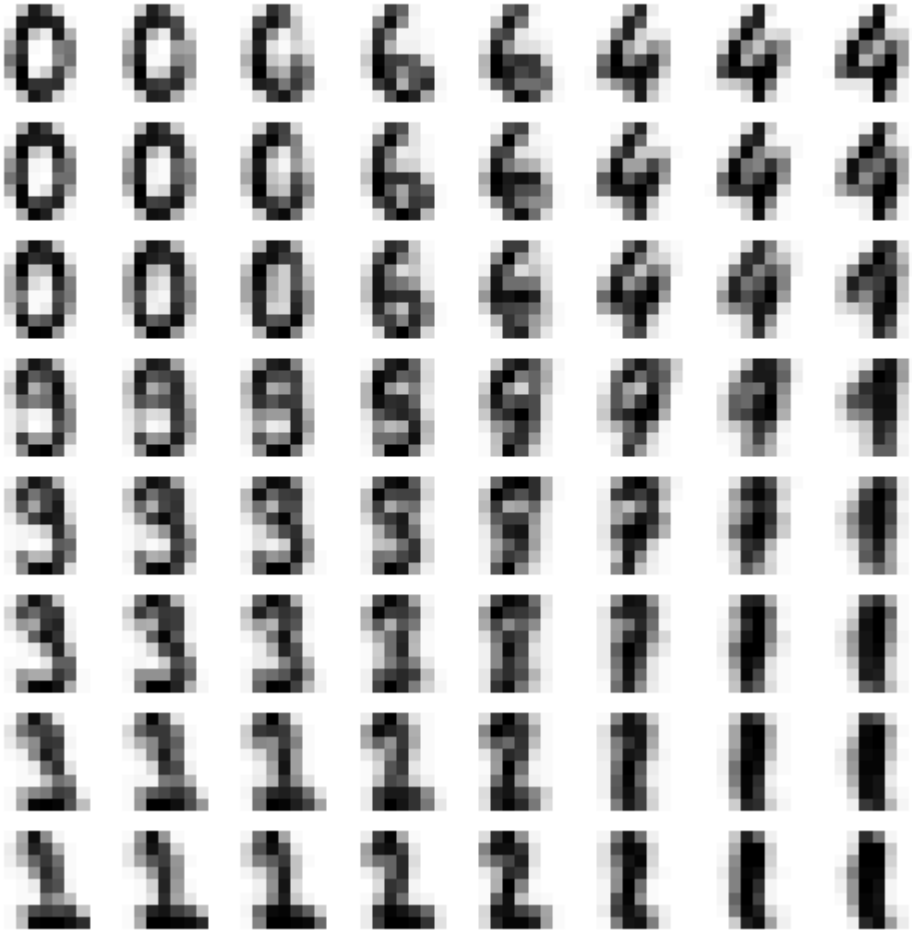
```
torch.manual_seed(0)
class TinyVAE(nn.Module):
    def __init__(self, z_dim=2):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(64, 128), nn.ReLU(), nn.Linear(128, 2 * z_dim))
        self.dec = nn.Sequential(nn.Linear(z_dim, 128), nn.ReLU(), nn.Linear(128, 64),
nn.Sigmoid())
    def forward(self, x):
        mu, logvar = self.enc(x).chunk(2, dim=1)
        z = mu + torch.randn_like(mu) * torch.exp(0.5 * logvar)        # sample a code
        return self.dec(z), mu, logvar

vae = TinyVAE(); opt = torch.optim.Adam(vae.parameters(), lr=2e-3)
Xall = torch.tensor(X / 16, dtype=torch.float32)
for epoch in range(300):
    perm = torch.randperm(len(Xall))
    for i in range(0, len(perm), 128):
        xb = Xall[perm[i:i + 128]]
        recon, mu, logvar = vae(xb)
        rec_loss = nn.functional.binary_cross_entropy(recon, xb, reduction="sum")
        kl = -0.5 * torch.sum(1 + logvar - mu ** 2 - logvar.exp())          # keep codes bell-
shaped
        loss = (rec_loss + kl) / len(xb); opt.zero_grad(); loss.backward(); opt.step()

# GENERATE: random codes in, new digits out
with torch.no_grad():
    grid = torch.tensor([[a, b] for b in np.linspace(2, -2, 8) for a in np.linspace(-2, 2, 8)],
dtype=torch.float32)
    new_digits = vae.dec(grid).numpy().reshape(-1, 8, 8)
fig, axes = plt.subplots(8, 8, figsize=(6, 6))
for ax, im in zip(axes.ravel(), new_digits):
```

```
    ax.imshow(im, cmap="gray_r"); ax.axis("off")
fig.suptitle("64 NEW digits generated from a grid of 2-number codes (none of them is a copy of a
training image)", fontsize=9)
plt.show()
```

64 NEW digits generated from a grid of 2-number codes (none of them is a copy of a training image)



```
# Prove they are new: distance from each generated digit to its NEAREST training image
gen = new_digits.reshape(64, -1); train_pix = X / 16
nearest = np.sqrt(((gen[:, None, :] - train_pix[None, :, :]) ** 2).sum(-1)).min(axis=1)
typical = np.sqrt(((train_pix[:200, None, :] - train_pix[None, :, :]) ** 2).sum(-1))
np.fill_diagonal(typical[:, :200], np.inf)
print(f"generated digit -> nearest training digit: median distance {np.median(nearest):.2f}")
print(f"real digit -> its nearest OTHER real digit: median distance
{np.median(typical.min(axis=1)):.2f}")
```

```
generated digit -> nearest training digit: median distance 0.99
real digit -> its nearest OTHER real digit: median distance 1.05
```

- The generated digits are **plausible but new**: they're about as far from their nearest training image as real digits are from each other, so the model isn't simply copying.
- Moving smoothly across the grid morphs one digit into another, because the model has learned a continuous "space of digits".
- They're blurry. A 2-number code and an 8×8 world is a toy. ChatGPT, Claude, Midjourney and other real GenAI systems apply the same "learn the data distribution, then sample from it" idea with **billions** of parameters and **web-scale** data, mostly with **Transformer** and diffusion architectures, as the lecture says.

The four levels side by side (this notebook's evidence)

```
print(f"{'level':<10}{'approach':<42}{'features chosen by':<22}{'result'}")
print(f"{'AI':<10}{'hand-written rules':<42}{'a human (rules)':<22}{np.mean(pred ==
y_te[mask]):.0%} on 0-vs-1 only")
print(f"{'ML':<10}{'logistic regression, engineered features':<42}{'a human (features)':<22}
{ml.score(F_te, y_te):.0%} on all 10 digits")
print(f"{'DL':<10}{'2-hidden-layer network, raw pixels':<42}{'the network itself':<22}
{dl_acc:.0%} on all 10 digits")
print(f"{'GenAI':<10}{'tiny VAE':<42}{'the network itself':<22}creates new digits")
```

```
level     approach                                  features chosen by    result
AI        hand-written rules                        a human (rules)       97% on 0-vs-1 only
ML        logistic regression, engineered features  a human (features)    74% on all 10 digits
DL        2-hidden-layer network, raw pixels        the network itself    98% on all 10 digits
GenAI     tiny VAE                                  the network itself    creates new digits
```

✅ Check yourself

1. Put these in the hierarchy: a chess program with hand-coded rules, a Naive Bayes spam filter, a CNN tagging beach photos, ChatGPT writing a poem. *(AI only; AI+ML; AI+ML+DL; AI+ML+DL+GenAI)*
2. What is feature engineering, and at which level does it stop being manual? *(choosing which measurements the model sees; DL learns them automatically)*
3. Why did the hand-engineered-feature model stay worst even with all the training data? *(its 8 features discard information that more data can't recover; the model is limited by the features a human chose)*
4. What makes a model *generative* rather than *discriminative*? *(it models the data itself and can produce new samples, rather than only mapping inputs to labels)*
5. "GenAI is a completely different technology from DL." True or false? *(False: it's deep learning applied at scale to content generation)*

02 · From the biological neuron to backpropagation (1943 → 1986), in code

Lectures: *History of Deep Learning* (04-08-2026) and *The Biological Neuron Analogy* (05-08-2026, Part B) · **Notes:** `../notes/01_history_of_deep_learning.md`, `../notes/02_ai_ml_dl_genai_and_the_neuron.md`

The history lecture is a timeline. Here we **run** its first four milestones:

Year	Milestone	What we code
1943	McCulloch & Pitts: the first artificial neuron	a threshold neuron that computes AND, OR and NOT, with **no learning**
1958	Rosenblatt: the perceptron	the same neuron, but its weights **learn** from examples (synaptic plasticity)
1969	Minsky & Papert: the limits	the perceptron **fails on XOR**, which led to the first "AI winter"
1986	Rumelhart, Hinton & Williams: backpropagation	a **multi-layer** network trained by backpropagation **solves XOR**

Along the way we map each part of the code to the **biological neuron** (dendrites, synapses, soma, threshold, axon). Runs in seconds.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
```

The mapping we'll keep using

Biological neuron	Artificial neuron	In the code below
dendrites receive signals	**inputs** x_1, x_2, ...	`x`
synapse strength	**weights** w_1, w_2, ...	`w`
soma integrates (sums) the signals	**weighted sum + bias**	`z = w @ x + b`
firing threshold / action potential (all-or-nothing)	**activation function**	`1 if z >= 0 else 0`
axon carries the output on	**output** to the next layer	the return value
synaptic plasticity (learning)	**weight updates**	`w += ...` (from 1958 on)

1943 · The McCulloch–Pitts neuron: logic from a threshold

Binary inputs, fixed weights, and a threshold θ: **fire (1) if the weighted sum reaches θ**. McCulloch and Pitts showed that such neurons can compute logic, which suggested that a brain-like network could, in principle, compute anything.

```
def mcculloch_pitts(x, w, theta):
    soma = np.dot(w, x)                 # integrate the incoming signals
    return int(soma >= theta)           # all-or-nothing firing

gates = {
    "AND": dict(w=[1, 1], theta=2),     # fires only if BOTH inputs fire
    "OR":  dict(w=[1, 1], theta=1),     # fires if AT LEAST ONE input fires
}
for name, p in gates.items():
    print(name, [mcculloch_pitts(x, **p) for x in X])
print("NOT", [mcculloch_pitts([x], w=[-1], theta=0) for x in [0, 1]], " (an inhibitory input:
weight -1)")
```

```
AND [0, 0, 0, 1]
OR [0, 1, 1, 1]
NOT [1, 0]  (an inhibitory input: weight -1)
```

No learning happened: **a human chose** the weights and thresholds. That's 1943's limitation, and it's what the **perceptron** changed.

1958 · Rosenblatt's perceptron: the weights learn

Same neuron (weights, sum, threshold), plus a **learning rule** that adjusts the weights after every mistake: **w ← w + η·(y − ŷ)·x**. This is the artificial version of **synaptic plasticity**: connections that should have fired more get stronger. (`03_perceptron_from_scratch.ipynb` covers this in depth.)

```
def train_perceptron(X, y, eta=0.1, epochs=50):
    w, b, history = np.zeros(2), 0.0, []
    for epoch in range(epochs):
        mistakes = 0
        for xi, yi in zip(X, y):
            y_hat = int(np.dot(w, xi) + b >= 0)
            w += eta * (yi - y_hat) * xi        # synaptic plasticity
            b += eta * (yi - y_hat)
            mistakes += int(yi != y_hat)
        history.append(mistakes)
        if mistakes == 0:
            break
    return w, b, history

for name, y in [("AND", np.array([0, 0, 0, 1])), ("OR", np.array([0, 1, 1, 1]))]:
    w, b, hist = train_perceptron(X, y)
    print(f"{name}: learned w = {w}, b = {b:.1f} in {len(hist)} epochs -> outputs {[int(np.dot(w,
x) + b >= 0) for x in X]}")
```

```
AND: learned w = [0.2 0.1], b = -0.2 in 4 epochs -> outputs [0, 0, 0, 1]
OR: learned w = [0.1 0.1], b = -0.1 in 4 epochs -> outputs [0, 1, 1, 1]
```

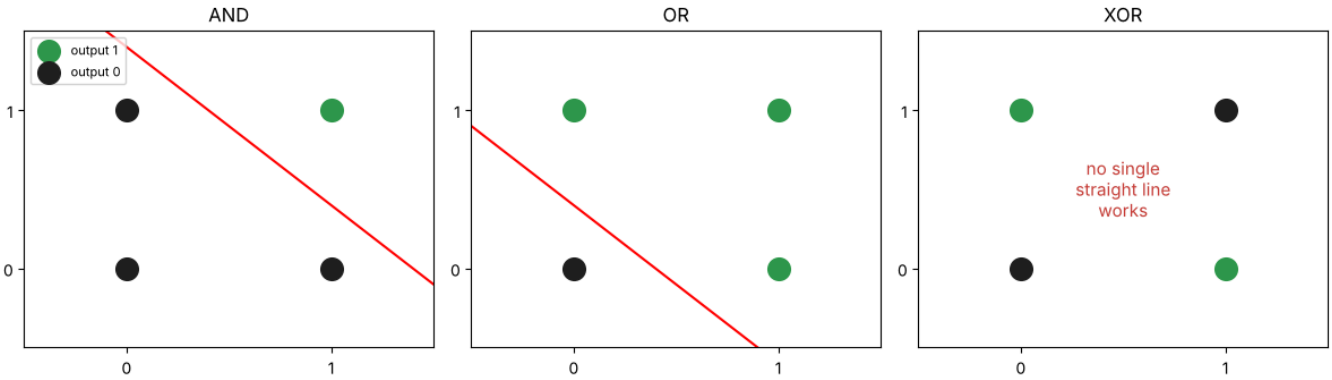
The perceptron **found weights for AND and OR by itself**. That was the excitement of 1958: a machine that learns from examples.

1969 · Minsky & Papert: the XOR problem → the AI winter

```
y_xor = np.array([0, 1, 1, 0])
w, b, hist = train_perceptron(X, y_xor, epochs=50)
print("XOR mistakes per epoch:", hist[:12], "... still", hist[-1], "mistakes after", len(hist),
"epochs")
print("outputs:", [int(np.dot(w, x) + b >= 0) for x in X], " target:", y_xor.tolist())

fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
for ax, (name, yy) in zip(axes, [("AND", [0, 0, 0, 1]), ("OR", [0, 1, 1, 1]), ("XOR", [0, 1, 1,
0])]):
    yy = np.array(yy)
    ax.scatter(*X[yy == 1].T, s=200, c="#2e9a4b", label="output 1"); ax.scatter(*X[yy == 0].T,
s=200, c="#222", label="output 0")
    ax.set_xlim(-0.5, 1.5); ax.set_ylim(-0.5, 1.5); ax.set_title(name); ax.set_xticks([0, 1]);
ax.set_yticks([0, 1])
axes[0].plot([-0.5, 1.5], [1.9, -0.1], "r-"); axes[1].plot([-0.5, 1.2], [0.9, -0.8], "r-")
axes[2].text(0.5, 0.5, "no single\nstraight line\nworks", ha="center", va="center",
color="#c8403a", fontsize=11)
axes[0].legend(loc="upper left", fontsize=8); plt.tight_layout(); plt.show()
```

```
XOR mistakes per epoch: [3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4] ... still 4 mistakes after 50 epochs
outputs: [1, 1, 0, 0]  target: [0, 1, 1, 0]
```



A single perceptron draws **one straight line**. XOR's 1s sit on one diagonal and its 0s on the other, so no line separates them, and the perceptron keeps making mistakes forever. Minsky and Papert's book *Perceptrons* (1969) made this limitation famous, funding dried up, and the field entered the **AI winter**: researchers moved to rule-based expert systems.

1986 · Backpropagation: multi-layer networks learn XOR

The fix needed two things:

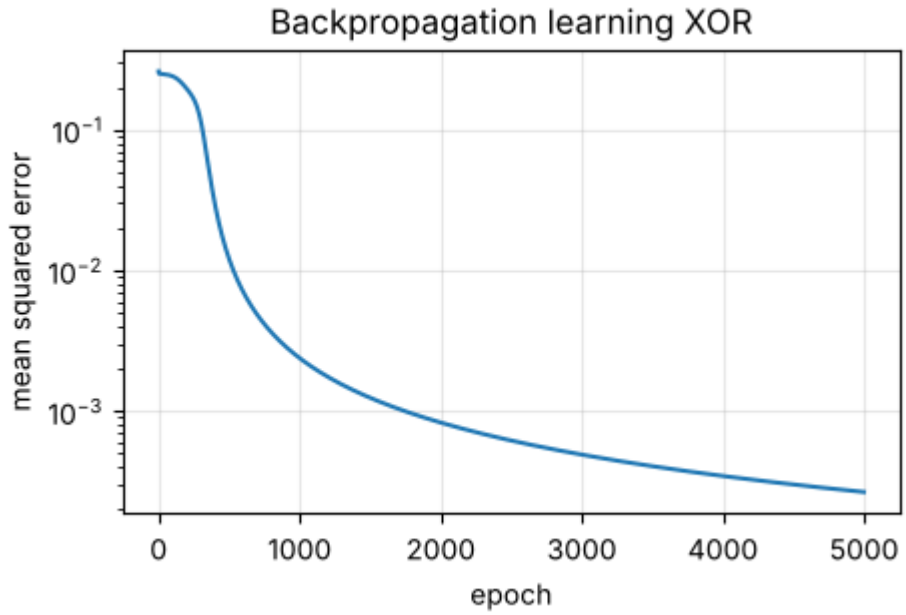
1. **Hidden layers**, so the network can bend its decision boundary.
2. A way to **train** them. The step function's slope is 0, so it gives no learning signal, and nobody knew how much each hidden neuron was to blame for an error.

Backpropagation (Rumelhart, Hinton & Williams, 1986) solved part 2: use a **smooth** activation (sigmoid), and pass the error **backwards** layer by layer with the chain rule, giving every weight its share of the blame. Here it is from scratch in NumPy (no deep-learning library), so nothing is hidden.

```
rng = np.random.default_rng(1)
sigmoid = lambda z: 1 / (1 + np.exp(-z))
W1, b1 = rng.normal(0, 1, (2, 3)), np.zeros(3)      # input (2) -> hidden (3)
W2, b2 = rng.normal(0, 1, (3, 1)), np.zeros(1)      # hidden (3) -> output (1)
Y = y_xor.reshape(-1, 1); eta = 1.0; losses = []
for epoch in range(5000):
    # forward pass: signals flow input -> hidden -> output (dendrites -> soma -> axon -> next
```

```
neuron)
    H = sigmoid(X @ W1 + b1)
    out = sigmoid(H @ W2 + b2)
    losses.append(np.mean((out - Y) ** 2))
    # backward pass: the error flows output -> hidden, each weight gets its share of the blame
    d_out = (out - Y) * out * (1 - out)              # chain rule through the output sigmoid
    d_hid = (d_out @ W2.T) * H * (1 - H)             # ... and back through the hidden sigmoids
    W2 -= eta * H.T @ d_out;  b2 -= eta * d_out.sum(0)
    W1 -= eta * X.T @ d_hid;  b1 -= eta * d_hid.sum(0)
print("outputs after training:", out.ravel().round(3), "-> rounded", (out.ravel() >
0.5).astype(int).tolist(), " target", y_xor.tolist())
plt.figure(figsize=(5, 3)); plt.plot(losses); plt.yscale("log"); plt.xlabel("epoch");
plt.ylabel("mean squared error")
plt.title("Backpropagation learning XOR"); plt.grid(alpha=0.3); plt.show()
```

```
outputs after training: [0.016 0.988 0.981 0.017] -> rounded [0, 1, 1, 0]  target [0, 1, 1, 0]
```



XOR solved. The two-layer network learned it from examples using backpropagation, which became the standard training algorithm and remains so today, inside every PyTorch `loss.backward()`.

What happened next (the rest of the timeline)

Year	Milestone	Where you meet it
1998	**LeNet-5** (LeCun): CNNs read handwritten digits and bank cheques	CV course Week 2–3
2006	**Deep Belief Networks** (Hinton): layer-wise pre-training makes deep nets trainable	the start of "modern" deep learning
2012	**AlexNet** (Krizhevsky, Sutskever, Hinton): GPUs + **ReLU** + **Dropout** win ImageNet	notebooks 04 and 05 here
2014	**GANs** (Goodfellow): generator vs discriminator	CV course Week 4
2017	**Transformers** ("Attention Is All You Need", Google)	the basis of LLMs
2020–	**Foundation models & Generative AI** (GPT, Llama, Gemini, Claude, Stable Diffusion)	notebook 01's tiny generative model is the toy version

Where the biological analogy stops

The code above captures the **computational principle** (weighted inputs → sum → threshold → output, with adjustable connections). It ignores almost everything else about real neurons:

	Biological	Our code
signal	electrochemical spikes, neurotransmitters	floating-point numbers
timing	asynchronous, continuous time	synchronous steps
learning	biochemical plasticity, not fully understood	an exact formula (backpropagation + gradient descent)
scale	~86 billion neurons, ~100 trillion synapses, ~20 watts	4 neurons (3 hidden + 1 output) and 13 learnable numbers here

✅ Check yourself

1. Match: dendrites, synapse, soma, firing threshold, axon, plasticity → ? *(inputs, weights, weighted sum + bias, activation function, output, weight updates)*
2. What could the 1943 neuron do, and what couldn't it do? *(compute logic with hand-set weights; it couldn't learn)*
3. What did the perceptron add in 1958? *(a learning rule that adjusts the weights from examples)*
4. Why did XOR cause an "AI winter"? *(a single-layer perceptron provably can't learn it, which undermined confidence and funding)*
5. What two things were needed to solve XOR, and which 1986 algorithm provided the training part? *(hidden layers + a way to train them; backpropagation with smooth activations)*

03 · The perceptron, from scratch

Lecture: *Perceptron* (06-08-2026) · **Notes:** `../notes/03_perceptron.md`

In this notebook you will:

1. Recompute the lecture's two worked examples in code (forward pass z = 1.4, and the weight update).
2. Build a `Perceptron` class in about 15 lines of NumPy and train it on the lecture's **Pass/Fail** example (hours of study x1, hours of practice x2).
3. Watch the decision boundary move as the perceptron learns.
4. Test the lecture's claims about the **learning rate**.
5. Prove the famous limitation: AND and OR are learnable, **XOR is not**. Then fix XOR with a tiny multi-layer network.

Everything runs in a few seconds. Change the numbers and re-run: that's the point.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
np.set_printoptions(precision=3, suppress=True)
rng = np.random.default_rng(0)
```

1 · The two building blocks: weighted sum and step activation

Symbol	Meaning (lecture's notation)
x1, x2	inputs: hours of study, hours of practice
w1, w2	weights: how important each input is
b	bias: shifts the threshold
z	net input, **z = w1·x1 + w2·x2 + b**
f(z)	**step** activation: 1 if z ≥ 0, else 0
ŷ (y_hat)	prediction: 1 = Pass, 0 = Fail
y	the true answer
η (eta)	learning rate

```
def step(z):
    """Heaviside step: 1 if z >= 0 else 0 (works on numbers and arrays)."""
    return (np.asarray(z) >= 0).astype(int)

def forward(x, w, b):
    z = np.dot(w, x) + b          # weighted sum (net input)
    return z, step(z)             # (z, prediction)

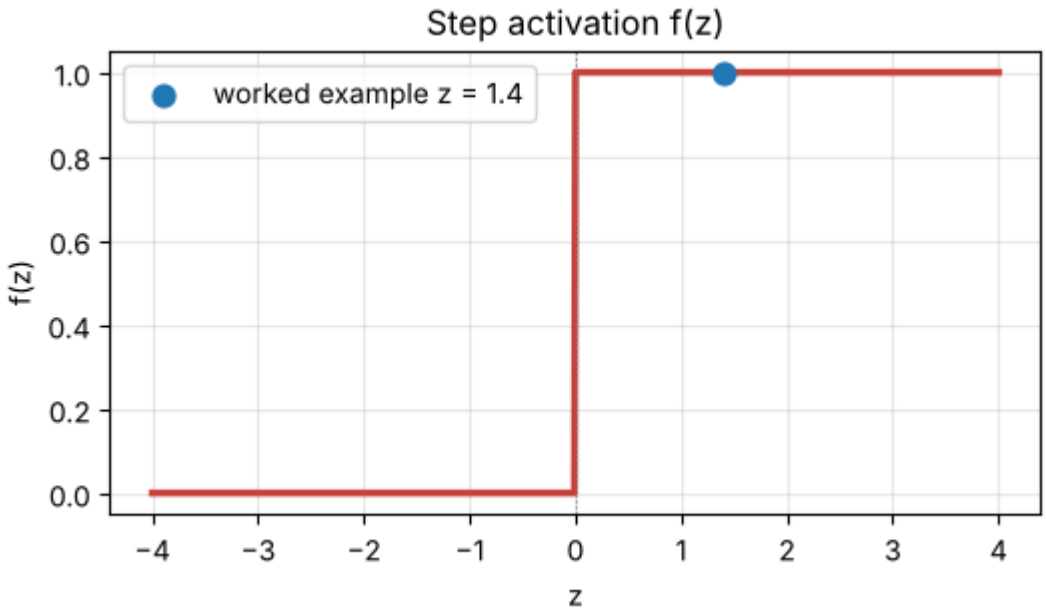
# Lecture worked example 3.2: x1=2, x2=3, w1=0.5, w2=0.8, b=-2
z, y_hat = forward(np.array([2, 3]), np.array([0.5, 0.8]), -2)
print(f"z = 2*0.5 + 3*0.8 + (-2) = {z:.1f}  ->  f(z) = {y_hat}  ({'Pass' if y_hat else 'Fail'})")
```

```
z = 2*0.5 + 3*0.8 + (-2) = 1.4  ->  f(z) = 1  (Pass)
```

That matches the lecture: **z = 1.4, ŷ = 1 (Pass)**.

The step function plotted. Note the jump at z = 0: at exactly z = 0 the lecture's rule says "fire" (1).

```
zs = np.linspace(-4, 4, 801)
plt.figure(figsize=(6, 3))
plt.plot(zs, step(zs), lw=2.5, color="#c8403a")
plt.axvline(0, color="grey", lw=0.8, ls=":")
plt.scatter([1.4], [1], s=60, zorder=3, label="worked example z = 1.4")
plt.title("Step activation f(z)"); plt.xlabel("z"); plt.ylabel("f(z)"); plt.legend();
plt.grid(alpha=0.3)
plt.show()
```



2 · Learning: the perceptron update rule

Forward propagation only **predicts**. To **learn**, compare the prediction with the truth and nudge the weights:

$w_i \leftarrow w_i + \eta \cdot (y - \hat{y}) \cdot x_i$ $b \leftarrow b + \eta \cdot (y - \hat{y})$

The error $(y - \hat{y})$ can only be −1, 0 or +1:

- **0**: the prediction was right, so nothing changes.
- **+1**: it said Fail but the answer was Pass (under-prediction), so the weights move **up** in proportion to each input.
- **−1**: it said Pass but the answer was Fail (over-prediction), so the weights move **down**.

Lecture worked example 5.2: x = (2, 3), w = (0.2, −0.4), b = −1.0, y = 1, ŷ = 0, η = 0.1.

```
def update(x, y, y_hat, w, b, eta):
    error = y - y_hat
    return w + eta * error * x, b + eta * error

w_new, b_new = update(np.array([2, 3]), 1, 0, np.array([0.2, -0.4]), -1.0, 0.1)
print("new weights:", w_new, " new bias:", round(b_new, 2))

# does the updated perceptron now get it right?
```

```
z_before, p_before = forward(np.array([2, 3]), np.array([0.2, -0.4]), -1.0)
z_after, p_after = forward(np.array([2, 3]), w_new, b_new)
print(f"before: z = {z_before:.2f} -> {p_before}   after one update: z = {z_after:.2f} ->
{p_after}")
```

```
new weights: [ 0.4 -0.1]  new bias: -0.9
before: z = -1.80 -> 0   after one update: z = -0.40 -> 0
```

Matches the lecture: **w1 = 0.4, w2 = −0.1, b = −0.9**. Notice two things the lecture doesn't spell out:

- One update moved z from −1.8 to −0.4. That is **closer** to the threshold, but the prediction is **still wrong** (0). Learning takes several passes over the data.
- Each weight moved **in proportion to its input** (x2 = 3 moved w2 more than x1 = 2 moved w1). Inputs that were "louder" get more of the blame.

3 · A `Perceptron` class, trained on Pass/Fail data

Made-up but realistic data: 60 students. A student passes if 0.6·study + 0.4·practice is above 5, a hidden rule the perceptron has to discover. Borderline students (score within 0.4 of 5) are left out, so there is a clear gap between Pass and Fail.

```
class Perceptron:
    def __init__(self, n_inputs, eta=0.1, w_init=None):
        self.w = np.zeros(n_inputs) if w_init is None else np.array(w_init, float)
        self.b = 0.0
        self.eta = eta
        self.history = []                       # (w, b, mistakes) after every epoch

    def predict(self, X):
        return step(X @ self.w + self.b)

    def fit(self, X, y, epochs=20, verbose=False):
        for epoch in range(epochs):
            mistakes = 0
            for xi, yi in zip(X, y):            # one example at a time (online learning)
                y_hat = step(xi @ self.w + self.b)
                error = yi - y_hat
                self.w += self.eta * error * xi
                self.b += self.eta * error
                mistakes += int(error != 0)
            self.history.append((self.w.copy(), self.b, mistakes))
            if verbose and (epoch < 5 or mistakes == 0 or (epoch + 1) % 10 == 0):
                print(f"epoch {epoch + 1:2d}: mistakes = {mistakes:2d}   w = {self.w}  b =
{self.b:.2f}")
            if mistakes == 0:                   # converged: every training example correct
                break
        return self

# the data
study = rng.uniform(0, 10, 200)
practice = rng.uniform(0, 10, 200)
score = 0.6 * study + 0.4 * practice
keep = np.abs(score - 5) > 0.4            # drop borderline students so a clear gap exists
study, practice, score = study[keep][:60], practice[keep][:60], score[keep][:60]
n = len(study)
y = (score > 5).astype(int)
X = np.column_stack([study, practice])
print("students:", n, "  passed:", y.sum(), "  failed:", n - y.sum())

p = Perceptron(2, eta=0.1).fit(X, y, epochs=500, verbose=True)
print("training accuracy:", (p.predict(X) == y).mean())
```

```
students: 60   passed: 29   failed: 31
epoch  1: mistakes = 31   w = [ 0.46  -0.832]  b = -1.10
epoch  2: mistakes = 21   w = [ 0.777 -0.61 ]  b = -1.60
epoch  3: mistakes = 22   w = [ 0.586 -0.275]  b = -2.20
epoch  4: mistakes = 22   w = [ 0.836 -0.388]  b = -2.80
epoch  5: mistakes = 21   w = [ 0.846 -0.088]  b = -3.30
epoch 10: mistakes = 16   w = [ 0.891 -0.048]  b = -5.50
epoch 19: mistakes =  0   w = [0.835 0.714]  b = -7.60
training accuracy: 1.0
```

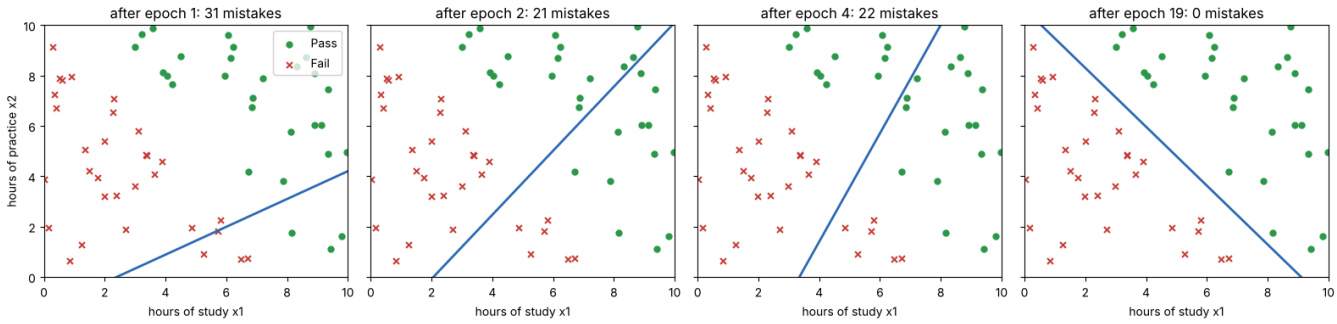
The number of mistakes per epoch shrinks to **0**: the perceptron found a line that separates every Pass from every Fail. The **perceptron convergence theorem** guarantees this happens in a finite number of steps **if** such a line exists (the data is linearly separable).

Watching the decision boundary move

The boundary is the line where z = 0: w1·x1 + w2·x2 + b = 0, i.e. x2 = −(w1·x1 + b)/w2.

```
def draw_boundary(ax, w, b, **kw):
    xs = np.array([0, 10])
    if abs(w[1]) > 1e-9:
        ax.plot(xs, -(w[0] * xs + b) / w[1], **kw)
    elif abs(w[0]) > 1e-9:
        ax.axvline(-b / w[0], **kw)

epochs_to_show = [0, 1, 3, len(p.history) - 1]
fig, axes = plt.subplots(1, 4, figsize=(16, 4), sharey=True)
for ax, e in zip(axes, epochs_to_show):
    w_e, b_e, m_e = p.history[e]
    ax.scatter(*X[y == 1].T, c="#2e9a4b", label="Pass", s=25)
    ax.scatter(*X[y == 0].T, c="#c8403a", label="Fail", s=25, marker="x")
    draw_boundary(ax, w_e, b_e, color="#2f6db5", lw=2)
    ax.set_xlim(0, 10); ax.set_ylim(0, 10)
    ax.set_title(f"after epoch {e + 1}: {m_e} mistakes"); ax.set_xlabel("hours of study x1")
axes[0].set_ylabel("hours of practice x2"); axes[0].legend(loc="upper right")
plt.tight_layout(); plt.show()
```



4 · Does the learning rate matter? (testing the lecture's claims)

The lecture says: η too small → slow; η too large → overshoot/diverge. That is true for **gradient descent** on a smooth loss (section 5 shows it). For the classic perceptron there is a surprise worth knowing:

```
for eta in [0.001, 0.01, 0.1, 1.0, 10.0]:
    pz = Perceptron(2, eta=eta).fit(X, y, epochs=200)          # weights start at zero
    pr = Perceptron(2, eta=eta, w_init=[-1.0, 1.0]).fit(X, y, epochs=200)   # weights start
elsewhere
```

```
    print(f"eta = {eta:>6}:  start at zero -> {len(pz.history):3d} epochs  |  start at (-1, 1) ->
{len(pr.history):3d} epochs")
```

```
eta =  0.001:  start at zero ->  19 epochs  |  start at (-1, 1) ->  19 epochs
eta =   0.01:  start at zero ->  19 epochs  |  start at (-1, 1) ->  13 epochs
eta =    0.1:  start at zero ->  19 epochs  |  start at (-1, 1) ->  13 epochs
eta =    1.0:  start at zero ->  19 epochs  |  start at (-1, 1) ->  18 epochs
eta =   10.0:  start at zero ->  19 epochs  |  start at (-1, 1) ->   3 epochs
```

What the numbers show: when the weights start at **zero**, every η takes **exactly the same number of epochs**. The step function only looks at the **sign** of z, and multiplying all the weights and the bias by η doesn't change any sign. So η just rescales the whole solution. When the weights start somewhere else, η does matter, because it sets how big the corrections are compared with the starting weights.

The lecture's learning-rate trade-off is really about **gradient descent on a smooth loss**, which modern networks use. Here it is:

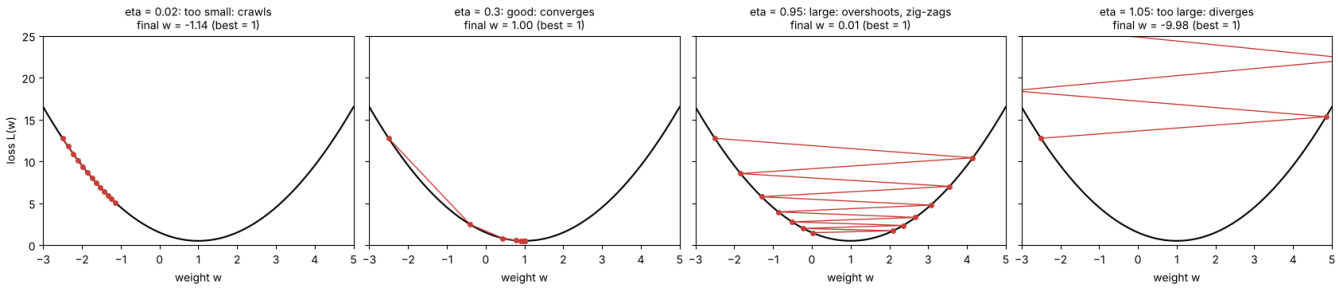
5 · Gradient descent: walking downhill on a loss curve

A loss curve $L(w) = (w - 1)^2 + 0.5$. Its slope is $2(w - 1)$. Each step: **$w \leftarrow w - \eta \cdot \text{slope}$**.

```
L = lambda w: (w - 1) ** 2 + 0.5
dL = lambda w: 2 * (w - 1)

def descend(w0, eta, steps=12):
    ws = [w0]
    for _ in range(steps):
        ws.append(ws[-1] - eta * dL(ws[-1]))
    return np.array(ws)

ws_grid = np.linspace(-3, 5, 400)
fig, axes = plt.subplots(1, 4, figsize=(17, 3.8), sharey=True)
for ax, (eta, title) in zip(axes, [(0.02, "too small: crawls"), (0.3, "good: converges"),
                                   (0.95, "large: overshoots, zig-zags"), (1.05, "too large:
diverges")]):
    path = descend(-2.5, eta)
    ax.plot(ws_grid, L(ws_grid), color="black", lw=1.5)
    ax.plot(path, L(path), "o-", color="#c8403a", ms=4, lw=1)
    ax.set_title(f"eta = {eta}: {title}\nfinal w = {path[-1]:.2f} (best = 1)", fontsize=10)
    ax.set_ylim(0, 25); ax.set_xlim(-3, 5); ax.set_xlabel("weight w")
axes[0].set_ylabel("loss L(w)")
plt.tight_layout(); plt.show()
```



This confirms the lecture's trade-off: tiny steps crawl, good steps arrive quickly, large steps zig-zag across the valley, and **too-large steps make the loss grow** (diverge).

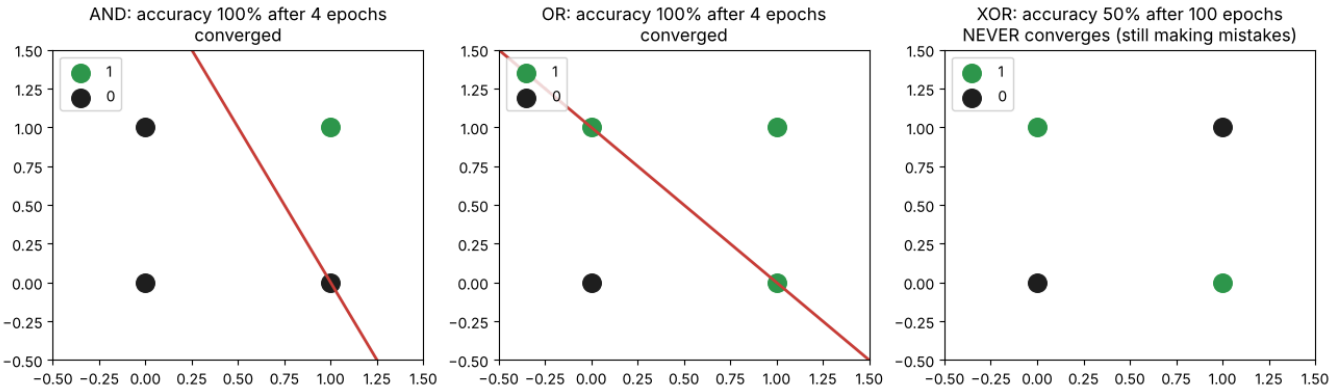
> **Local vs global minimum:** this loss has only one valley, so any converging η finds the global minimum. Real neural-network losses have many valleys and flat regions, and there the step size (and luck) affect where you end up.

6 · Linear separability: AND ✓, OR ✓, XOR ×

```
X_gate = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
gates = {"AND": np.array([0, 0, 0, 1]), "OR": np.array([0, 1, 1, 1]), "XOR": np.array([0, 1, 1,
0])}

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, (name, yg) in zip(axes, gates.items()):
    pg = Perceptron(2, eta=0.1).fit(X_gate, yg, epochs=100)
    acc = (pg.predict(X_gate) == yg).mean()
    converged = pg.history[-1][2] == 0
    ax.scatter(*X_gate[yg == 1].T, s=160, c="#2e9a4b", label="1")
    ax.scatter(*X_gate[yg == 0].T, s=160, c="#222", label="0")
    xs = np.array([-0.5, 1.5])
    w_g, b_g = pg.w, pg.b
    if abs(w_g[1]) > 1e-9:
        ax.plot(xs, -(w_g[0] * xs + b_g) / w_g[1], color="#c8403a", lw=2)
    ax.set_xlim(-0.5, 1.5); ax.set_ylim(-0.5, 1.5)
    ax.set_title(f"{name}: accuracy {acc:.0%} after {len(pg.history)} epochs\n"
                 f"{'converged' if converged else 'NEVER converges (still making mistakes)'}")
    ax.legend(loc="upper left")
    print(f"{name}: weights {pg.w}, bias {pg.b:.1f}, accuracy {acc:.0%}, converged =
{converged}")
plt.tight_layout(); plt.show()
```

```
AND: weights [0.2 0.1], bias -0.2, accuracy 100%, converged = True
OR: weights [0.1 0.1], bias -0.1, accuracy 100%, converged = True
XOR: weights [-0.1  0. ], bias 0.0, accuracy 50%, converged = False
```



A closer look: the AND line passes right through a point

Look at the AND panel: the learned line goes **exactly through (1, 0)**. Let's print z for every input:

```
p_and = Perceptron(2, eta=0.1).fit(X_gate, gates["AND"], epochs=100)
for x in X_gate:
    z = x @ p_and.w + p_and.b
    print(f"x = {x}: z = {z:+.20f} -> prediction {step(z)}")
```

```
x = [0 0]: z = -0.20000000000000003886 -> prediction 0
x = [0 1]: z = -0.10000000000000003331 -> prediction 0
x = [1 0]: z = -0.00000000000000002776 -> prediction 0
x = [1 1]: z = +0.10000000000000000555 -> prediction 1
```

For (1, 0), z is a tiny number from floating-point rounding, not a real margin. Mathematically 0.2 + (−0.2) = 0, which the step function would call **1**, a wrong answer for AND. The run is correct only because of a rounding hair.

Two lessons:

- The perceptron **stops as soon as it makes no mistakes**, however close a point sits to the line. It doesn't look for a *safe* boundary.
- Algorithms that maximise the **margin** (the gap between the line and the nearest points), like the **Support Vector Machine**, were designed to fix exactly this. A boundary with a wide margin generalises better to new, slightly different inputs.

```
# Can ANY single line do better on XOR? Try 200,000 random lines and keep the best.
W = rng.normal(size=(200_000, 2)); B = rng.normal(size=200_000)
preds = step(X_gate @ W.T + B)                          # shape (4, 200000)
best = (preds == gates["XOR"][:, None]).mean(axis=0).max()
print(f"best accuracy of any of 200,000 random lines on XOR: {best:.0%}")
```

```
best accuracy of any of 200,000 random lines on XOR: 75%
```

No straight line gets more than 3 of the 4 XOR points right (75%). This is the limitation Minsky and Papert published in 1969, and it helped start the first "AI winter".

7 · The fix: add a hidden layer (a multi-layer perceptron)

XOR = (x1 OR x2) AND NOT (x1 AND x2). Two perceptrons in a hidden layer compute OR and NAND, and one output perceptron ANDs them. **Hand-set weights:**

```
def mlp_xor(x):
    h_or   = step(x @ np.array([1, 1]) - 0.5)        # OR
    h_nand = step(x @ np.array([-1, -1]) + 1.5)      # NAND = NOT(AND)
    return step(h_or + h_nand - 1.5)                 # AND of the two hidden neurons

for x in X_gate:
    print(x, "->", mlp_xor(x))
```

```
[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 0
```

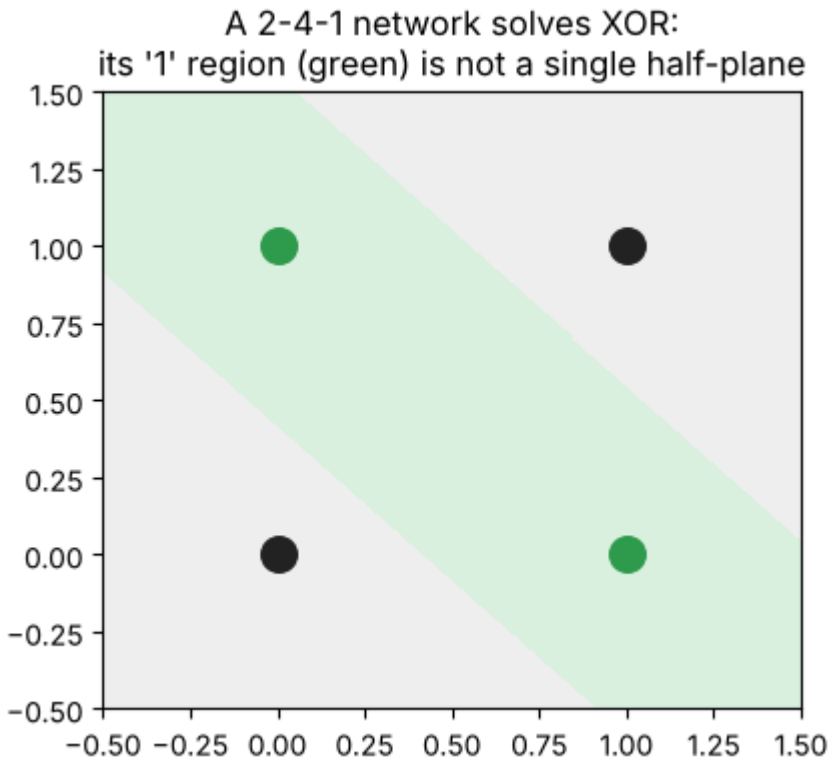
Now let a network **learn** XOR by itself. The step function's slope is zero everywhere, so gradient descent can't train it. We swap in a smooth activation (sigmoid, see notebook 04) and use **backpropagation**, the 1986 algorithm that ended the AI winter.

```
import torch, torch.nn as nn
torch.manual_seed(0)
Xt = torch.tensor(X_gate, dtype=torch.float32); yt = torch.tensor(gates["XOR"],
dtype=torch.float32).unsqueeze(1)
net = nn.Sequential(nn.Linear(2, 4), nn.Sigmoid(), nn.Linear(4, 1), nn.Sigmoid())   # 2 -> 4
hidden -> 1
opt = torch.optim.SGD(net.parameters(), lr=1.0)
for epoch in range(3000):
    loss = nn.functional.binary_cross_entropy(net(Xt), yt)
    opt.zero_grad(); loss.backward(); opt.step()
print("final loss:", round(loss.item(), 4))
print("outputs:", net(Xt).detach().squeeze().numpy().round(3), "-> rounded:", (net(Xt) >
0.5).int().squeeze().tolist())

# the curved decision region it learned
g = np.linspace(-0.5, 1.5, 200); GX, GY = np.meshgrid(g, g)
grid = torch.tensor(np.column_stack([GX.ravel(), GY.ravel()]), dtype=torch.float32)
Z = net(grid).detach().numpy().reshape(GX.shape)
plt.figure(figsize=(4.5, 4))
plt.contourf(GX, GY, Z > 0.5, levels=1, colors=["#eeeeee", "#d9f0df"])
plt.scatter(*X_gate[gates["XOR"] == 1].T, s=160, c="#2e9a4b"); plt.scatter(*X_gate[gates["XOR"]
== 0].T, s=160, c="#222")
```

```
plt.title("A 2-4-1 network solves XOR:\nits '1' region (green) is not a single half-plane");
plt.show()
```

```
final loss: 0.003
outputs: [0.003 0.997 0.997 0.003] -> rounded: [0, 1, 1, 0]
```



The hidden layer lets the network **bend** its decision region, which a single line can't do. That is the whole reason for "deep" networks.

✅ Check yourself

1. Compute by hand: x = (1, 4), w = (0.5, −0.5), b = 1. What are z and ŷ? *(z = 0.5 − 2 + 1 = −0.5 → ŷ = 0)*
2. The perceptron predicted 1 but the answer is 0, with η = 0.5 and x = (2, 1). How does each weight change? *(error = −1 → w1 −1.0, w2 −0.5, b −0.5)*
3. Why does the perceptron stop updating once it gets everything right? *(error = 0 for every example, so the update is 0)*
4. Why can't a single perceptron learn XOR, and what fixes it? *(one straight line can't separate XOR's diagonal pattern; a hidden layer, i.e. an MLP, bends the boundary)*
5. Why can't you train a step-function network with gradient descent? *(the step's slope is 0 everywhere except the jump, so there's no gradient signal; use smooth activations like sigmoid or ReLU)*

Try this: add noise so a few Pass/Fail labels are wrong (flip 3 labels) and re-run section 3. Does the perceptron still converge? *(No. The data is no longer linearly separable, so it keeps making mistakes, just like XOR.)*

04 · Activation functions: see them, break them, compare them

Lectures: *Activation Functions in Neural Networks* (07-08-2026: Step, Linear) and *(Cont.)* (11-08-2026: Sigmoid, tanh, ReLU, Leaky ReLU, PReLU) · **Notes:** `../notes/04_activation_functions.md`

In this notebook you will:

1. Plot all seven activations **and their derivatives** (the derivative is what training uses).
2. **Prove** that stacking linear layers is pointless without a non-linear activation.
3. Measure the **vanishing-gradient** problem of sigmoid and tanh.
4. Create **dying ReLU** neurons on purpose, and watch Leaky ReLU and PReLU avoid them.
5. Train the same network with each activation and compare honestly.

Runs in about 1–2 minutes on a laptop CPU.

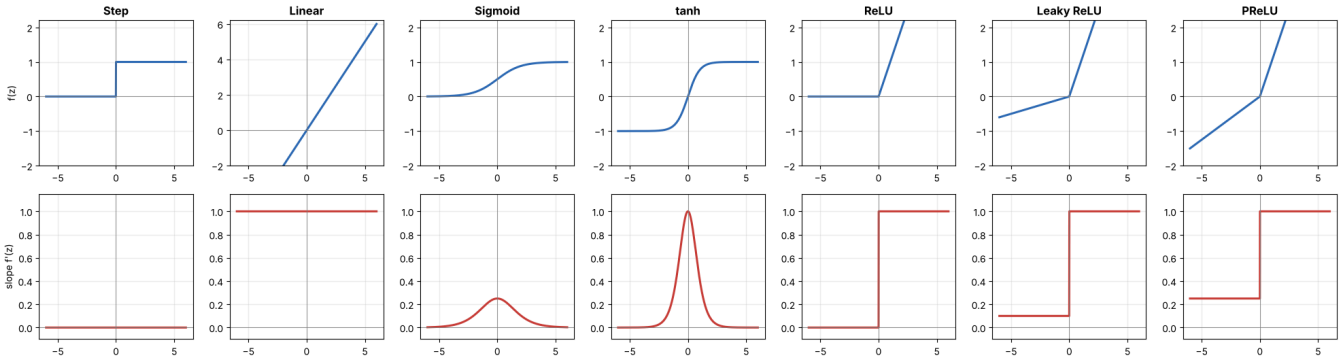
```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
torch.manual_seed(0); np.random.seed(0)
```

1 · The seven functions and their slopes

Function	Formula	Derivative (slope)
Step	1 if z ≥ 0 else 0	0 (undefined at 0)
Linear	z	1
Sigmoid σ	$1 / (1 + e^{-z})$	$\sigma(z)\cdot(1 - \sigma(z)) \le 0.25$
tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	$1 - \tanh^2(z) \le 1$
ReLU	max(0, z)	1 if z > 0, else 0
Leaky ReLU	z if z > 0, else α·z (α fixed, e.g. 0.01)	1 or α
PReLU	z if z > 0, else a·z (**a learned**)	1 or a

```
z = np.linspace(-6, 6, 1201)
sig = 1 / (1 + np.exp(-z))
alpha, a_prelu = 0.1, 0.25      # alpha exaggerated (usual is 0.01) so the leak is visible; 0.25
= PyTorch's PReLU start value
funcs = {
    "Step":       ((z >= 0).astype(float),          np.zeros_like(z)),
    "Linear":     (z,                               np.ones_like(z)),
    "Sigmoid":    (sig,                             sig * (1 - sig)),
    "tanh":       (np.tanh(z),                      1 - np.tanh(z) ** 2),
    "ReLU":       (np.maximum(0, z),                (z > 0).astype(float)),
    "Leaky ReLU": (np.where(z > 0, z, alpha * z),   np.where(z > 0, 1, alpha)),
    "PReLU":      (np.where(z > 0, z, a_prelu * z), np.where(z > 0, 1, a_prelu)),
}
fig, axes = plt.subplots(2, 7, figsize=(20, 5.5))
```

```
for j, (name, (f, d)) in enumerate(funcs.items()):
    axes[0, j].plot(z, f, lw=2.2, color="#2f6db5"); axes[0, j].set_title(name, fontweight="bold")
    axes[1, j].plot(z, d, lw=2.2, color="#c8403a")
    for ax in axes[:, j]:
        ax.axhline(0, color="grey", lw=0.6); ax.axvline(0, color="grey", lw=0.6);
ax.grid(alpha=0.25)
    axes[0, j].set_ylim(-2, 2.2 if name != "Linear" else 6.2)
    axes[1, j].set_ylim(-0.1, 1.15)
axes[0, 0].set_ylabel("f(z)"); axes[1, 0].set_ylabel("slope f'(z)")
plt.tight_layout(); plt.show()
print("max sigmoid slope:", round((sig * (1 - sig)).max(), 4), " max tanh slope:", round((1 -
np.tanh(z) ** 2).max(), 4))
```




```
max sigmoid slope: 0.25  max tanh slope: 1.0
```

Read the **bottom row**. During training, backpropagation multiplies these slopes together layer after layer:

- **Step:** slope 0 everywhere, so there is no learning signal (why the perceptron rule isn't gradient descent).
- **Sigmoid:** slope at most **0.25**, and near 0 once $|z| > 4$ (saturation).
- **tanh:** slope up to **1** at zero, but also near 0 once saturated.
- **ReLU:** slope exactly **1** for positive z, but **0** for negative z (the dying-ReLU risk).
- **Leaky ReLU / PReLU:** never zero, so a small signal always gets through.

Useful values: $\sigma(0) = 0.5$ and $\tanh(0) = 0$. **tanh is zero-centred, sigmoid is not.**

```
for v in [-3, -1, 0, 1, 3]:
    s = 1 / (1 + np.exp(-v))
    print(f"z = {v:>2}:  sigmoid = {s:.3f}  tanh = {np.tanh(v):+.3f}  ReLU = {max(0, v)}
LeakyReLU(0.01) = {v if v > 0 else 0.01 * v:+.2f}")
```

```
z = -3:  sigmoid = 0.047  tanh = -0.995  ReLU = 0  LeakyReLU(0.01) = -0.03
z = -1:  sigmoid = 0.269  tanh = -0.762  ReLU = 0  LeakyReLU(0.01) = -0.01
z =  0:  sigmoid = 0.500  tanh = +0.000  ReLU = 0  LeakyReLU(0.01) = +0.00
z =  1:  sigmoid = 0.731  tanh = +0.762  ReLU = 1  LeakyReLU(0.01) = +1.00
z =  3:  sigmoid = 0.953  tanh = +0.995  ReLU = 3  LeakyReLU(0.01) = +3.00
```

2 · Why linear layers alone are not enough (a proof in code)

Two linear layers: **W2·(W1·x + b1) + b2 = (W2·W1)·x + (W2·b1 + b2)**. That's just **one** linear layer. However many you stack, the network can only draw straight lines.

```
torch.manual_seed(1)
deep_linear = nn.Sequential(nn.Linear(2, 16), nn.Linear(16, 16), nn.Linear(16, 16), nn.Linear(16,
1))  # 4 layers, NO activation
# Collapse it into ONE equivalent linear layer
W, b = torch.eye(2), torch.zeros(2)
```

```
for layer in deep_linear:
    W, b = layer.weight @ W, layer.weight @ b + layer.bias
x = torch.randn(5, 2)
print("4-layer linear network:", deep_linear(x).detach().squeeze().numpy().round(4))
print("single collapsed layer:", (x @ W.T + b).detach().squeeze().numpy().round(4))
print("identical:", torch.allclose(deep_linear(x), x @ W.T + b, atol=1e-5))
```

```
4-layer linear network: [-0.1407 -0.2472 -0.2158 -0.044  -0.3272]
single collapsed layer: [-0.1407 -0.2472 -0.2158 -0.044  -0.3272]
identical: True
```

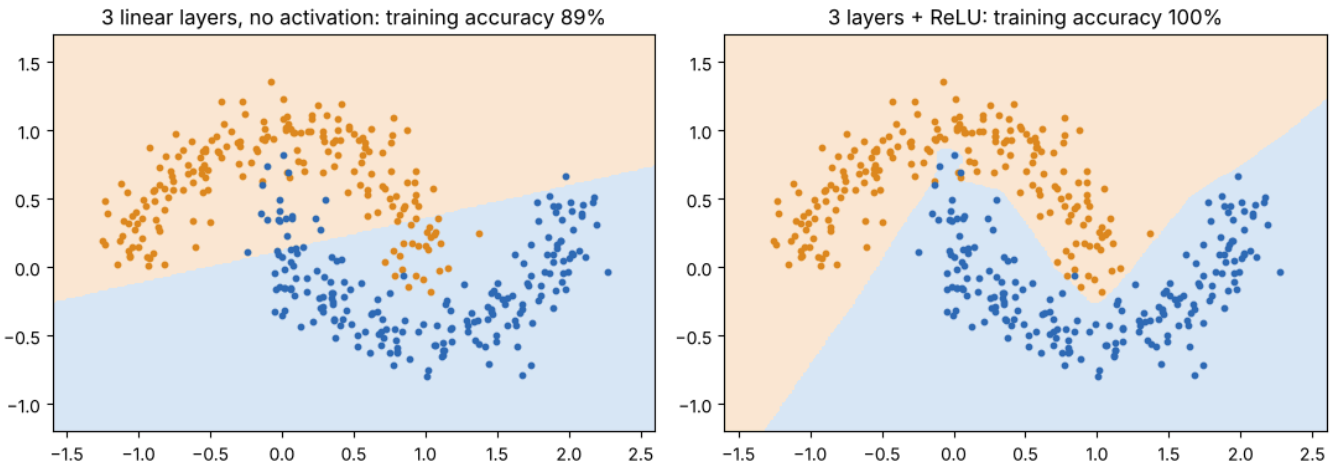
Now the practical consequence: train a deep **linear** network and a deep **ReLU** network on data that needs a curved boundary (two interleaving half-moons).

```
from sklearn.datasets import make_moons
Xm, ym = make_moons(400, noise=0.15, random_state=0)
Xm_t, ym_t = torch.tensor(Xm, dtype=torch.float32), torch.tensor(ym,
dtype=torch.float32).unsqueeze(1)

def make_net(act, depth=3, width=32):
    layers, d_in = [], 2
    for _ in range(depth):
        layers += [nn.Linear(d_in, width)] + ([act()] if act else [])
        d_in = width
    return nn.Sequential(*layers, nn.Linear(d_in, 1))

def train_bin(net, X, y, epochs=600, lr=0.01):
    opt = torch.optim.Adam(net.parameters(), lr=lr)
    for _ in range(epochs):
        loss = nn.functional.binary_cross_entropy_with_logits(net(X), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return ((net(X) > 0).float() == y).float().mean().item()

g = np.linspace(-1.6, 2.6, 200); GX, GY = np.meshgrid(g, np.linspace(-1.2, 1.7, 200))
grid = torch.tensor(np.column_stack([GX.ravel(), GY.ravel()]), dtype=torch.float32)
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, (name, act) in zip(axes, [("3 linear layers, no activation", None), ("3 layers + ReLU",
nn.ReLU)]):
    torch.manual_seed(0); net = make_net(act)
    acc = train_bin(net, Xm_t, ym_t)
    Z = (net(grid) > 0).float().detach().numpy().reshape(GX.shape)
    ax.contourf(GX, GY, Z, levels=1, colors=["#fbe9d2", "#dbe7f6"])
    ax.scatter(*Xm[ym == 0].T, s=10, c="#e08a1e"); ax.scatter(*Xm[ym == 1].T, s=10, c="#2f6db5")
    ax.set_title(f"{name}: training accuracy {acc:.0%}")
plt.tight_layout(); plt.show()
```



Without an activation, the "deep" network can only draw a **straight line**. With ReLU it bends around the moons.

3 · Vanishing gradients: measured

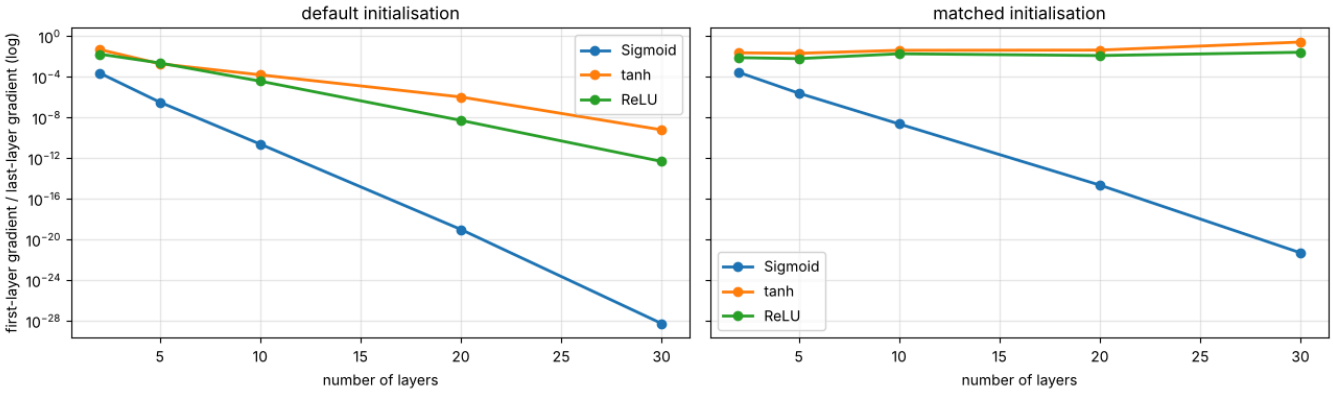
A deep stack of layers. We send one batch through it, backpropagate, and measure how big the gradient is at the **first** layer, compared with the last layer. We try two ways of choosing the starting weights:

- **PyTorch's default** initialisation, used for every activation;
- the **matching** initialisation that practitioners pair with each activation: **He (Kaiming)** for ReLU and **Xavier (Glorot)** for sigmoid and tanh. These scale the random starting weights so the signal neither shrinks nor explodes from layer to layer.

```
def first_layer_grad(act, depth, init="default", width=64, seed=0):
    torch.manual_seed(seed)
    layers = []
    for _ in range(depth):
        lin = nn.Linear(width, width)
        if init == "matched":
            if act is nn.ReLU:
                nn.init.kaiming_normal_(lin.weight, nonlinearity="relu")          # He
initialisation
            else:
                gain = nn.init.calculate_gain("tanh" if act is nn.Tanh else "sigmoid")
                nn.init.xavier_normal_(lin.weight, gain=gain)                     # Xavier
initialisation
            nn.init.zeros_(lin.bias)
        layers += [lin, act()]
    net = nn.Sequential(*layers, nn.Linear(width, 1))
    x = torch.randn(256, width)
    net(x).pow(2).mean().backward()
    return net[0].weight.grad.abs().mean().item() / net[-1].weight.grad.abs().mean().item()

depths = [2, 5, 10, 20, 30]
acts = {"Sigmoid": nn.Sigmoid, "tanh": nn.Tanh, "ReLU": nn.ReLU}
fig, axes = plt.subplots(1, 2, figsize=(13, 4), sharey=True)
for ax, init in zip(axes, ["default", "matched"]):
    for name, a in acts.items():
        r = [first_layer_grad(a, d, init) for d in depths]
        ax.semilogy(depths, r, "o-", label=name, lw=2)
        print(f"{init:>8} init, {name:>7}: " + "  ".join(f"{d} layers {v:.0e}" for d, v in
zip(depths, r)))
    ax.set_title(f"{init} initialisation"); ax.set_xlabel("number of layers"); ax.grid(alpha=0.3,
which="both"); ax.legend()
axes[0].set_ylabel("first-layer gradient / last-layer gradient (log)")
plt.tight_layout(); plt.show()
```

```
 default init, Sigmoid: 2 layers 2e-04  5 layers 3e-07  10 layers 2e-11  20 layers 1e-19  30 laye
rs 5e-29
 default init,    tanh: 2 layers 5e-02  5 layers 2e-03  10 layers 2e-04  20 layers 1e-06  30 laye
rs 6e-10
 default init,    ReLU: 2 layers 2e-02  5 layers 2e-03  10 layers 4e-05  20 layers 5e-09  30 laye
rs 5e-13
 matched init, Sigmoid: 2 layers 3e-04  5 layers 2e-06  10 layers 2e-09  20 layers 2e-15  30 laye
rs 5e-22
 matched init,    tanh: 2 layers 2e-02  5 layers 2e-02  10 layers 4e-02  20 layers 4e-02  30 laye
rs 3e-01
 matched init,    ReLU: 2 layers 8e-03  5 layers 6e-03  10 layers 2e-02  20 layers 1e-02  30 laye
rs 3e-02
```



What it shows:

- **Sigmoid vanishes either way.** With matched initialisation, the gradient reaching layer 1 of a 30-layer net is still only about 10^{-22} of the last layer's, so those early layers can't learn. Its slope is at most 0.25, and multiplying 30 such factors together is hopeless. This is the lecture's **vanishing-gradient problem**.
- **tanh and ReLU need the matching initialisation.** With PyTorch's default starting weights, both also lose most of the signal as depth grows (ReLU at least as badly as tanh here). With **He for ReLU and Xavier for tanh**, the gradient stays roughly level even at 30 layers.

> **Lesson:** "ReLU fixes vanishing gradients" is half the story. ReLU's slope of 1 *allows* the signal to survive, but only with sensible starting weights (He initialisation). Standard models pair them for you, and add normalisation layers (notebook 05).

4 · Dying ReLU

A ReLU neuron whose input is negative for **every** example outputs 0 and has slope 0. It gets **no gradient, ever again**: it is **dead**.

4a · One neuron, seen up close

We teach a single neuron the function y = x on inputs 0...1, starting from a badly negative bias (b = −2), so z = w·x + b < 0 for every input.

```
x1 = torch.linspace(0, 1, 50).unsqueeze(1); target = x1.clone()
for name, f in [("ReLU", torch.relu), ("Leaky ReLU (alpha 0.01)", lambda t:
nn.functional.leaky_relu(t, 0.01))]:
    w = torch.tensor([1.0], requires_grad=True); b = torch.tensor([-2.0], requires_grad=True)
    opt = torch.optim.SGD([w, b], lr=0.5)
    for step_i in range(2001):
        loss = ((f(x1 * w + b) - target) ** 2).mean()
        opt.zero_grad(); loss.backward()
        if step_i in (0, 2000):
            print(f"{name:>23} step {step_i:4d}: loss {loss.item():.4f}  bias {b.item():+.3f}
gradient on bias {b.grad.item():+.1e}")
        opt.step()
```

```
                  ReLU step    0: loss 0.3367  bias -2.000  gradient on bias +0.0e+00
                  ReLU step 2000: loss 0.3367  bias -2.000  gradient on bias +0.0e+00
Leaky ReLU (alpha 0.01) step    0: loss 0.3502  bias -2.000  gradient on bias -1.0e-02
Leaky ReLU (alpha 0.01) step 2000: loss 0.0000  bias -0.000  gradient on bias +6.3e-09
```

- **ReLU:** the gradient is **exactly 0**, at step 0 and still at step 2000. Nothing ever changes, and the neuron is permanently dead.
- **Leaky ReLU:** a small but non-zero gradient keeps nudging the bias up until the neuron comes back to life and fits the data (loss → 0).

4b · A whole network with most neurons knocked out

Imagine a bad update (or a bad start) has pushed every hidden bias to −0.35. We count the neurons that are never positive for any training image, then train with plain SGD.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
digits = load_digits()
Xd = digits.data / 16.0; yd = digits.target
Xtr, Xte, ytr, yte = train_test_split(Xd, yd, test_size=0.3, random_state=0, stratify=yd)
Xtr, Xte = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xte, dtype=torch.float32)
ytr, yte = torch.tensor(ytr), torch.tensor(yte)

def digit_net(act, depth=3, width=64, bias=None, init="default"):
    layers, d_in = [], 64
    for _ in range(depth):
        lin = nn.Linear(d_in, width)
        if init == "he":
            nn.init.kaiming_normal_(lin.weight, nonlinearity="relu"); nn.init.zeros_(lin.bias)
        if bias is not None:
            nn.init.constant_(lin.bias, bias)
        layers += [lin, act()]; d_in = width
    return nn.Sequential(*layers, nn.Linear(d_in, 10))

def train_digits(net, lr, epochs=40, opt_name="sgd"):
    opt = torch.optim.SGD(net.parameters(), lr=lr) if opt_name == "sgd" else
torch.optim.Adam(net.parameters(), lr=lr)
    for _ in range(epochs):
        perm = torch.randperm(len(Xtr))
        for i in range(0, len(Xtr), 64):
            idx = perm[i:i + 64]
            loss = nn.functional.cross_entropy(net(Xtr[idx]), ytr[idx])
            opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        return (net(Xte).argmax(1) == yte).float().mean().item()

def never_positive(net, X):
    """share of hidden neurons whose output is <= 0 for EVERY input"""
    count, total, h = 0, 0, X
    with torch.no_grad():
        for layer in net[:-1]:
            h = layer(h)
            if not isinstance(layer, nn.Linear):
                count += (h <= 0).all(dim=0).sum().item(); total += h.shape[1]
    return count / total

print(f"{'activation':>23} {'never positive: before':>23} {'after':>6} {'test acc':>9}")
for name, act in [("ReLU", nn.ReLU), ("Leaky ReLU (alpha 0.01)", lambda: nn.LeakyReLU(0.01)),
                  ("Leaky ReLU (alpha 0.1)", lambda: nn.LeakyReLU(0.1)), ("PReLU (learned
slope)", nn.PReLU)]:
    torch.manual_seed(0); net = digit_net(act, bias=-0.35)
    before = never_positive(net, Xtr)
    acc = train_digits(net, lr=0.1, epochs=60)
    print(f"{name:>23} {before:>23.0%} {never_positive(net, Xtr):>6.0%} {acc:>9.1%}")
```

```
             activation  never positive: before  after  test acc
                   ReLU                     77%    77%     10.2%
Leaky ReLU (alpha 0.01)                     77%    77%     10.2%
```

```
 Leaky ReLU (alpha 0.1)                    77%     77%      10.2%
  PReLU (learned slope)                    77%      0%      97.8%
```

Honest reading:

- **ReLU** stays stuck at chance (about 10% for 10 digits). Its dead neurons never recover.
- **Leaky ReLU with a fixed slope** does have a non-zero gradient, as the lecture says, and 4a showed that saves a single neuron. But here the signal must pass through **three** leaky layers (0.01 × 0.01 × 0.01), so with plain SGD and 60 epochs this network **also** stayed stuck, even with a 10× bigger slope (α = 0.1). "Reduces the risk" is not "removes the risk".
- **PReLU** learned to **change its slope** and escaped completely, reaching high accuracy. This is the flexibility the lecture credits it with.

PReLU's learned slopes (normal start, normal training)

```
torch.manual_seed(0); net = digit_net(nn.PReLU); acc = train_digits(net, 0.1)
slopes = [m.weight.item() for m in net if isinstance(m, nn.PReLU)]
print("PReLU negative slopes: start 0.25 each -> learned:", [round(s, 3) for s in slopes], f"
(test acc {acc:.1%})")
```

```
PReLU negative slopes: start 0.25 each -> learned: [1.794, 1.341, 2.062]   (test acc 94.8%)
```

Each PReLU layer has one extra trainable number (its negative-side slope `a`), starting at 0.25, and training moved them. A learned slope can even end up **above 1**: nothing forces it to stay small, so the network picks whatever helps. With `nn.PReLU(num_parameters=width)` you get one slope **per neuron**, which is more flexible but adds more parameters (the lecture's overfitting caveat).

5 · A fair comparison: the same network, each activation

A 6-hidden-layer network on digits, trained identically (Adam, lr 0.001, 30 epochs), averaged over 3 seeds. First all with **PyTorch's default initialisation**, then ReLU again with **He initialisation**.

```
for name, act in [("Sigmoid", nn.Sigmoid), ("tanh", nn.Tanh), ("ReLU", nn.ReLU), ("LeakyReLU",
lambda: nn.LeakyReLU(0.01)), ("PReLU", nn.PReLU)]:
    accs = []
    for seed in range(3):
        torch.manual_seed(seed); net = digit_net(act, depth=6)
        accs.append(train_digits(net, 1e-3, epochs=30, opt_name="adam"))
    print(f"{name:>15}: test accuracy {np.mean(accs):.1%} ± {np.std(accs):.1%}")
accs = []
for seed in range(3):
    torch.manual_seed(seed); net = digit_net(nn.ReLU, depth=6, init="he")
    accs.append(train_digits(net, 1e-3, epochs=30, opt_name="adam"))
print(f"{'ReLU + He init':>15}: test accuracy {np.mean(accs):.1%} ± {np.std(accs):.1%}")
```

```
        Sigmoid: test accuracy 27.3% ± 6.0%
           tanh: test accuracy 95.2% ± 0.7%
           ReLU: test accuracy 91.0% ± 2.4%
      LeakyReLU: test accuracy 93.1% ± 1.9%
          PReLU: test accuracy 94.6% ± 0.2%
 ReLU + He init: test accuracy 96.9% ± 0.5%
```

Honest reading of these numbers:

- **Sigmoid fails badly** at 6 layers (around 27%, near chance for 10 classes): that's the vanishing gradient in practice.
- With PyTorch's **default** initialisation, **tanh** (95.2%) actually beat plain **ReLU** (91.0%), with Leaky ReLU (93.1%) and PReLU (94.6%) in between.
- Give ReLU its matching **He initialisation** and it becomes the **best of all (96.9%)**, as section 3 predicted.
- **Takeaway:** "use ReLU" really means "use ReLU **with He initialisation**". Any ranking of activations depends on the setup (initialisation, optimiser, depth, data), so compare on your own validation data, as the lecture's summary says.

6 · Which activation where (lecture summary, now with evidence)

Situation	Use	Evidence in this notebook
hidden layers (default)	**ReLU** (+ He initialisation)	gradient stays level with He initialisation (section 3); best accuracy with He initialisation (section 5)
dying-ReLU concern	**Leaky ReLU**	revives a dead neuron (4a), but a small fixed slope may not rescue a badly broken deep net (4b)
want the leak learned	**PReLU**	learned its way out of 77% knocked-out neurons (4b)
bounded, zero-centred hidden state (RNN/LSTM)	**tanh**	tanh(0) = 0, range (−1, 1); strong with default initialisation (section 5)
binary classification output	**Sigmoid**	output in (0, 1), a probability; a poor hidden-layer choice (sections 3 and 5)
regression output	**Linear**	unbounded continuous output
classic perceptron only	**Step**	slope 0, so it can't be trained by gradient descent

✅ Check yourself

1. What is the maximum slope of sigmoid, and why does that cause vanishing gradients in deep networks? *(0.25; multiplied over many layers it shrinks towards 0)*
2. Why can't a 10-layer network with linear activations learn the moons? *(it collapses to one linear layer, so a straight boundary)*
3. Vanishing gradient vs dying ReLU: what's the difference? *(vanishing = slopes small but non-zero, from saturating sigmoid/tanh; dying ReLU = slope exactly 0 because the input is always negative)*
4. Leaky ReLU vs PReLU? *(Leaky: fixed slope α chosen by you; PReLU: slope learned during training)*
5. Which activation would you put on the output of a house-price model? A spam detector? *(Linear; Sigmoid)*
6. Why is "ReLU solves vanishing gradients" only half true? *(ReLU's slope of 1 lets the signal through, but only with suitable initialisation, i.e. He; with poor starting weights the signal still shrinks)*

05 · Normalization and regularization, by experiment

Lecture: *Normalization and Regularization in Neural Networks* (04-09-2026) · **Notes:** `../notes/05_normalization_and_regularization.md`

Two different problems, two different toolboxes:

	Normalization	Regularization
Problem it solves	badly **scaled** numbers make optimisation slow or unstable	the model **memorises** the training set (overfitting)
Goal	stable, fast training	better results on **unseen** data
Tools	standardisation, min-max, BatchNorm, LayerNorm	L1, L2, dropout, early stopping, augmentation, weight decay

In this notebook you will:

1. **See** gradient descent fail on unscaled features and fly on standardised ones.
2. Compare min-max and standardisation with an outlier, and see why test statistics must never leak in.
3. Compute **BatchNorm** and **LayerNorm** by hand and check them against PyTorch.
4. Make a network **overfit**, then try every regulariser from the lecture on it, measured honestly.

Runs in about 2–3 minutes on a laptop CPU.

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import torch, torch.nn as nn
rng = np.random.default_rng(0); torch.manual_seed(0)
```

```
<torch._C.Generator at 0x7f109ebd8dd0>
```

1 · Why scale matters: gradient descent on raw vs standardised features

Predict a price from two features with very different scales: **x1 = a rating from 0 to 1**, and **x2 = a floor area from 500 to 3000**. The true rule is y = 3·x1 + 0.002·x2 (+ noise). We fit it with plain gradient descent.

```
n = 200
x1 = rng.uniform(0, 1, n); x2 = rng.uniform(500, 3000, n)
y = 3 * x1 + 0.002 * x2 + rng.normal(0, 0.1, n)
X_raw = np.column_stack([x1, x2])

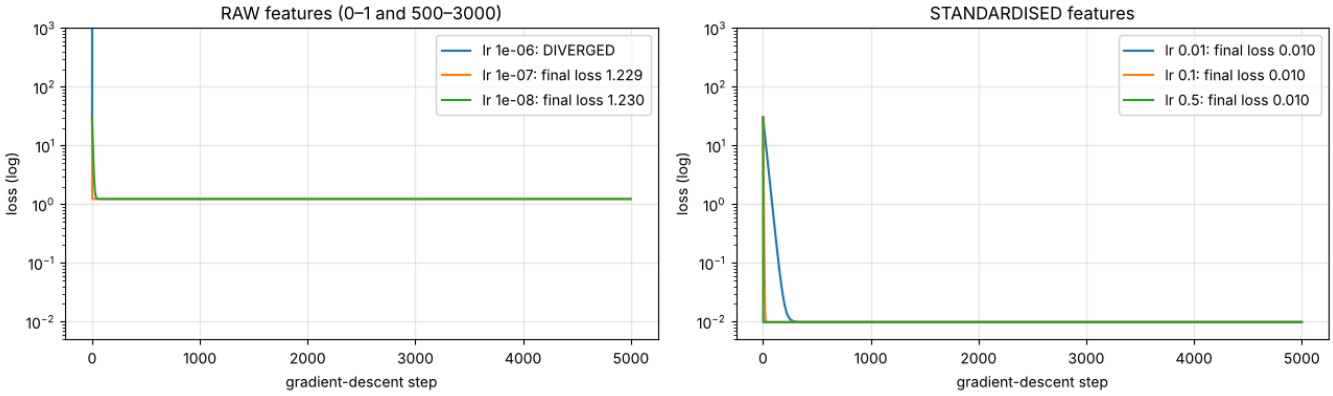
def gradient_descent(X, y, lr, steps=5000):
    w, b, losses = np.zeros(X.shape[1]), 0.0, []
    for _ in range(steps):
        err = X @ w + b - y
        loss = (err ** 2).mean()
```

```
        if not np.isfinite(loss) or loss > 1e6:
            losses.append(np.inf); break
        losses.append(loss)
        w -= lr * 2 * X.T @ err / len(y); b -= lr * 2 * err.mean()
    return np.array(losses)

mu, sigma = X_raw.mean(axis=0), X_raw.std(axis=0)
X_std = (X_raw - mu) / sigma                        # standardisation: (x - mean) / std

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
for lr in [1e-6, 1e-7, 1e-8]:
    L = gradient_descent(X_raw, y, lr)
    label = f"lr {lr:g}: " + ("DIVERGED" if np.isinf(L[-1]) else f"final loss {L[-1]:.3f}")
    axes[0].semilogy(L[np.isfinite(L)], label=label); print("raw         ", label)
for lr in [0.01, 0.1, 0.5]:
    L = gradient_descent(X_std, y, lr)
    first = int(np.argmax(L < 0.02)) if (L < 0.02).any() else None
    axes[1].semilogy(L, label=f"lr {lr:g}: final loss {L[-1]:.3f}"); print("standardised", f"lr
{lr:g}: final loss {L[-1]:.4f}, reached < 0.02 after {first} steps")
axes[0].set_title("RAW features (0–1 and 500–3000)"); axes[1].set_title("STANDARDISED features")
for ax in axes:
    ax.set_xlabel("gradient-descent step"); ax.set_ylabel("loss (log)"); ax.set_ylim(5e-3, 1e3);
ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

```
raw          lr 1e-06: DIVERGED
raw          lr 1e-07: final loss 1.229
raw          lr 1e-08: final loss 1.230
standardised lr 0.01: final loss 0.0098, reached < 0.02 after 199 steps
standardised lr 0.1: final loss 0.0098, reached < 0.02 after 19 steps
standardised lr 0.5: final loss 0.0098, reached < 0.02 after 1 steps
```



What happened:

- **Raw features:** the big feature x2 (hundreds to thousands) produces huge gradients, so the learning rate must be tiny to avoid **diverging**. But at that tiny rate the small feature x1 barely learns, and the loss **stalls**: the model is stuck far from the answer after 5,000 steps.
- **Standardised features:** both features are on the same scale, learning rates up to 100,000× larger are safe, and the loss reaches the noise floor in 199 steps (lr 0.01), 19 steps (lr 0.1) or even a single step (lr 0.5).

This is the lecture's point: normalization doesn't change **what** the model can learn, it makes the optimisation problem **easy to navigate**.

2 · Standardisation vs min-max, and data leakage

- **Standardisation (z-score):** $x' = (x - \mu) / \sigma$ → mean 0, standard deviation 1.
- **Min-max scaling:** $x' = (x - \min) / (\max - \min)$ → range [0, 1].

- **Images:** pixel / 255 → [0, 1].

Outliers: min-max's weakness

```
salaries = np.r_[rng.normal(50, 10, 99), 1000]           # 99 normal salaries (in thousands) +
one billionaire-style outlier
minmax = (salaries - salaries.min()) / (salaries.max() - salaries.min())
zscore = (salaries - salaries.mean()) / salaries.std()
print(f"min-max: the 99 normal values all lie in [{minmax[:99].min():.3f},
{minmax[:99].max():.3f}]  (squashed into {100 * (minmax[:99].max() - minmax[:99].min()):.0f}% of
the range)")
print(f"z-score: the 99 normal values lie in [{zscore[:99].min():.2f}, {zscore[:99].max():.2f}],
the outlier at {zscore[-1]:.1f}")
```

```
min-max: the 99 normal values all lie in [0.000, 0.049]  (squashed into 5% of the range)
z-score: the 99 normal values lie in [-0.35, 0.15], the outlier at 9.9
```

With min-max, one extreme value sets the range, so all the normal values get squashed into a tiny sliver near 0. Standardisation is affected too (the outlier shifts the mean and inflates σ), but less dramatically. For heavy outliers, people use robust scaling (median and interquartile range) or clip the outliers first.

Data leakage: fit the scaler on the training set ONLY

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
X_tr, X_te = train_test_split(X_raw, test_size=0.3, random_state=0)
right = StandardScaler().fit(X_tr)                  # learns mu and sigma from TRAIN data only
wrong = StandardScaler().fit(np.vstack([X_tr, X_te]))  # peeks at the test set
print("mean learned from train only:", right.mean_.round(3), "  from train+test:",
wrong.mean_.round(3))
X_tr_s, X_te_s = right.transform(X_tr), right.transform(X_te)   # SAME train statistics applied
to both
print("test set after correct scaling: mean", X_te_s.mean(axis=0).round(3), "(not exactly 0, and
that's fine)")
```

```
mean learned from train only: [5.360000e-01 1.763709e+03]   from train+test: [5.400000e-01 1.8043
61e+03]
test set after correct scaling: mean [0.045 0.192] (not exactly 0, and that's fine)
```

The rule from the lecture: **compute μ and σ (or min/max) from the training data, then apply those same numbers to validation and test data.** If test data helps choose the scaling, information about the test set has leaked into training, and the reported test score is no longer a fair "unseen data" score. Here the difference in μ is small, but in real projects (time series, small data, data from different hospitals) leakage can make results look much better than they really are.

3 · Normalization inside the network: BatchNorm and LayerNorm

BatchNorm by hand

For each **feature**, using the statistics of the **mini-batch**:

μ_B = mean over the batch · σ²_B = variance over the batch · x̂ = (x − μ_B) / √(σ²_B + ε) · y = γ·x̂ + β (γ, β learned)

```
torch.manual_seed(0)
batch = torch.randn(8, 4) * torch.tensor([1., 10., 0.1, 5.]) + torch.tensor([0., 50., -3., 7.])
# 8 samples, 4 features on wild scales
bn = nn.BatchNorm1d(4)                    # gamma starts at 1, beta at 0
bn.train()
out_torch = bn(batch)

mu_B = batch.mean(dim=0)                  # per FEATURE, across the batch
var_B = batch.var(dim=0, unbiased=False)
out_hand = (batch - mu_B) / torch.sqrt(var_B + bn.eps) * bn.weight + bn.bias
print("matches PyTorch:", torch.allclose(out_hand, out_torch, atol=1e-5))
print("per-feature mean after BN:", out_torch.mean(0).detach().numpy().round(3), " std:",
out_torch.std(0, unbiased=False).detach().numpy().round(3))
print("running mean kept for inference:", bn.running_mean.numpy().round(2), "(moved 10% of the
way towards this batch's mean, momentum 0.1)")
```

```
matches PyTorch: True
per-feature mean after BN: [-0.  0. -0.  0.]  std: [1.    1.    0.998 1.   ]
running mean kept for inference: [ 0.02  4.66 -0.3   0.67] (moved 10% of the way towards this bat
ch's mean, momentum 0.1)
```

Training mode vs inference mode

During training BatchNorm uses **the current batch's** statistics. At inference there may be just **one** example (no meaningful batch), so it uses the **running averages** it collected during training. That's why `model.eval()` matters:

```
single = batch[:1]
bn.train(); print("train mode, one sample:", bn(torch.cat([single, single])).detach()
[0].numpy().round(3), " <- batch of identical rows: variance 0, so everything becomes 0")
bn.eval();  print("eval mode,  one sample:", bn(single).detach()[0].numpy().round(3), " <- uses
running statistics")
```

```
train mode, one sample: [-0.  0.  0.  0.]  <- batch of identical rows: variance 0, so everything
becomes 0
eval mode,  one sample: [-1.09   7.508 -2.727  1.898]  <- uses running statistics
```

LayerNorm by hand

For each **sample**, using the statistics of **its own features**. It doesn't depend on the batch at all, which is why transformers and sequence models use it.

```
ln = nn.LayerNorm(4)
out_ln = ln(batch)
mu_L = batch.mean(dim=1, keepdim=True)              # per SAMPLE, across its features
var_L = batch.var(dim=1, unbiased=False, keepdim=True)
out_ln_hand = (batch - mu_L) / torch.sqrt(var_L + ln.eps) * ln.weight + ln.bias
print("matches PyTorch:", torch.allclose(out_ln_hand, out_ln, atol=1e-5))
print("each SAMPLE now has mean ~0:", out_ln.mean(1).detach().numpy().round(3))
print("same output for one sample alone as inside the batch:", torch.allclose(ln(batch[:1]),
out_ln[:1], atol=1e-6))
```

```
matches PyTorch: True
each SAMPLE now has mean ~0: [ 0. -0. -0.  0.  0. -0.  0.  0.]
same output for one sample alone as inside the batch: True
```

	BatchNorm	LayerNorm
averages over	the **batch** (per feature/channel)	the **features** (per sample)
depends on batch size	yes (very small batches give noisy statistics)	no
train vs eval behaviour	different (running statistics)	identical

	BatchNorm	LayerNorm
typical home	CNNs, feed-forward nets	transformers, RNNs

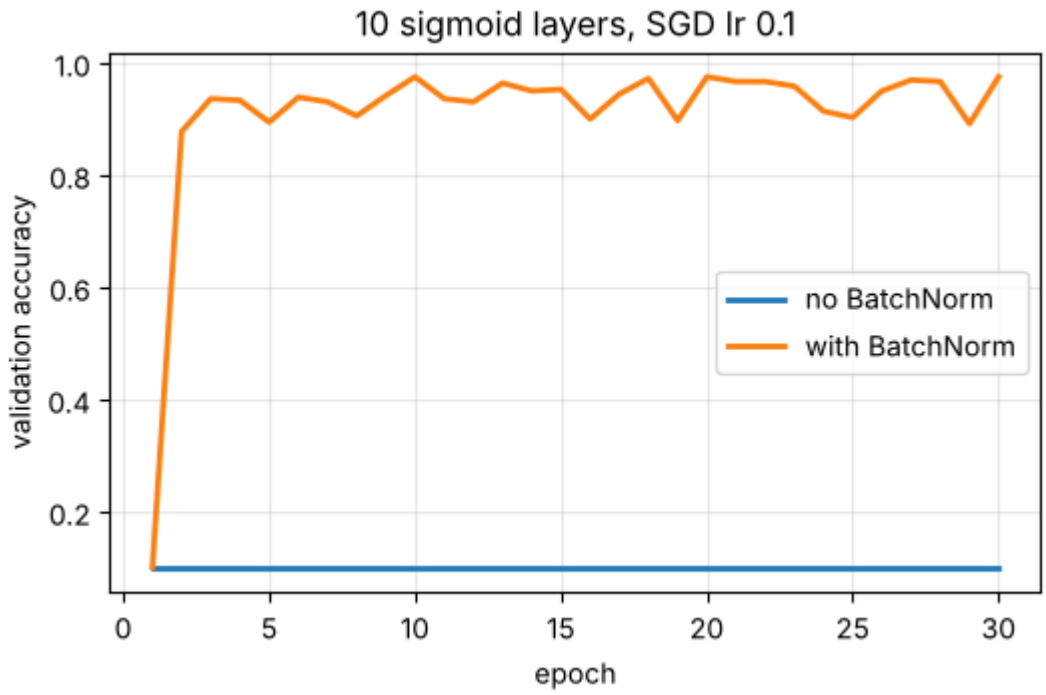
Does BatchNorm actually help training? A 10-layer network on digits

```
from sklearn.datasets import load_digits
digits = load_digits()
Xd, yd = digits.data / 16.0, digits.target       # image normalisation: pixels 0..16 -> 0..1
Xtr_all, Xtmp, ytr_all, ytmp = train_test_split(Xd, yd, test_size=0.4, random_state=0,
stratify=yd)
Xva_all, Xte_all, yva_all, yte_all = train_test_split(Xtmp, ytmp, test_size=0.5, random_state=0,
stratify=ytmp)
T = lambda a: torch.tensor(a, dtype=torch.float32)

def deep_net(use_bn, depth=10, width=64):
    layers, d_in = [], 64
    for _ in range(depth):
        layers += [nn.Linear(d_in, width)] + ([nn.BatchNorm1d(width)] if use_bn else []) +
[nn.Sigmoid()]
        d_in = width
    return nn.Sequential(*layers, nn.Linear(d_in, 10))

curves = {}
for use_bn in [False, True]:
    torch.manual_seed(0); net = deep_net(use_bn)
    opt = torch.optim.SGD(net.parameters(), lr=0.1)
    Xt, yt = T(Xtr_all), torch.tensor(ytr_all)
    accs = []
    for epoch in range(30):
        net.train()
        perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 32):
            idx = perm[i:i + 32]
            loss = nn.functional.cross_entropy(net(Xt[idx]), yt[idx]); opt.zero_grad();
loss.backward(); opt.step()
        net.eval()
        with torch.no_grad():
            accs.append((net(T(Xva_all)).argmax(1) ==
torch.tensor(yva_all)).float().mean().item())
    curves["with BatchNorm" if use_bn else "no BatchNorm"] = accs
    print(f"{'with' if use_bn else 'no  '} BatchNorm: validation accuracy after 5 epochs
{accs[4]:.1%}, after 30 epochs {accs[-1]:.1%}")
plt.figure(figsize=(6, 3.5))
for k, v in curves.items(): plt.plot(range(1, 31), v, label=k, lw=2)
plt.xlabel("epoch"); plt.ylabel("validation accuracy"); plt.title("10 sigmoid layers, SGD lr
0.1"); plt.legend(); plt.grid(alpha=0.3); plt.show()
```

```
no   BatchNorm: validation accuracy after 5 epochs 10.0%, after 30 epochs 10.0%
with BatchNorm: validation accuracy after 5 epochs 89.4%, after 30 epochs 97.5%
```



A deep sigmoid network is the textbook hard case (notebook 04 showed its gradients vanish). Without BatchNorm it **doesn't learn at all** in 30 epochs (10% = guessing among 10 digits). Inserting BatchNorm after every linear layer keeps each layer's inputs centred and scaled, so the sigmoids stay out of their flat regions, and the same network reaches about 89% after 5 epochs and 97.5% after 30.

4 · Regularization: make a network overfit, then fight it

To see overfitting clearly we make it easy to happen: only **200** training images, and a **big** network (64 → 512 → 512 → 10, about 300,000 weights). We keep a **validation** set to make decisions and a separate **test** set we look at only at the end.

```
Xtr, Xtmp, ytr, ytmp = train_test_split(Xd, yd, train_size=200, random_state=0, stratify=yd)
Xva, Xte, yva, yte = train_test_split(Xtmp, ytmp, test_size=0.5, random_state=0, stratify=ytmp)
Xtr, Xva, Xte = T(Xtr), T(Xva), T(Xte); ytr, yva, yte = torch.tensor(ytr), torch.tensor(yva),
torch.tensor(yte)
print("train", len(Xtr), " validation", len(Xva), " test", len(Xte))

def big_net(p_drop=0.0):
    return nn.Sequential(nn.Linear(64, 512), nn.ReLU(), nn.Dropout(p_drop),
                         nn.Linear(512, 512), nn.ReLU(), nn.Dropout(p_drop), nn.Linear(512, 10))

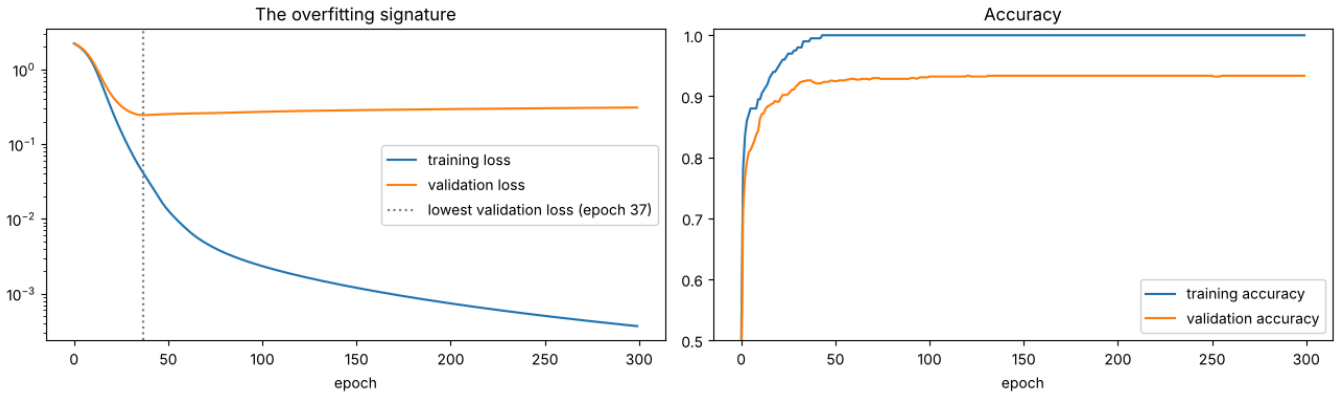
def evaluate(net, X, y):
    net.eval()
    with torch.no_grad():
        out = net(X); loss = nn.functional.cross_entropy(out, y).item(); acc = (out.argmax(1) ==
y).float().mean().item()
    net.train(); return loss, acc

def shift_1px(X):
    """data augmentation: move each 8x8 digit by up to 1 pixel (label unchanged)"""
    imgs = X.view(-1, 8, 8).clone()
    dx, dy = np.random.randint(-1, 2, size=(2, len(imgs)))
    for i in range(len(imgs)):
        imgs[i] = torch.roll(imgs[i], shifts=(int(dy[i]), int(dx[i])), dims=(0, 1))
    return imgs.view(-1, 64)
```

```python
def train_reg(p_drop=0.0, l1=0.0, l2=0.0, weight_decay=0.0, optimizer="adam", augment=False,
epochs=300, seed=0):
    torch.manual_seed(seed); np.random.seed(seed)
    net = big_net(p_drop)
    Opt = torch.optim.AdamW if optimizer == "adamw" else torch.optim.Adam
    opt = Opt(net.parameters(), lr=1e-3, weight_decay=weight_decay)
    hist = []
    for epoch in range(epochs):
        xb = shift_1px(Xtr) if augment else Xtr
        loss = nn.functional.cross_entropy(net(xb), ytr)
        weights = [m.weight for m in net if isinstance(m, nn.Linear)]
        if l1: loss = loss + l1 * sum(w.abs().sum() for w in weights)          # L1 penalty:
lambda * sum|w|
        if l2: loss = loss + l2 / 2 * sum((w ** 2).sum() for w in weights)     # L2 penalty:
lambda/2 * sum w^2
        opt.zero_grad(); loss.backward(); opt.step()
        hist.append(evaluate(net, Xtr, ytr) + evaluate(net, Xva, yva))         # (train loss,
train acc, val loss, val acc)
    return net, np.array(hist)

net0, h0 = train_reg()
best_epoch = int(h0[:, 2].argmin())
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(h0[:, 0], label="training loss"); axes[0].plot(h0[:, 2], label="validation loss")
axes[0].axvline(best_epoch, color="grey", ls=":", label=f"lowest validation loss (epoch
{best_epoch})")
axes[0].set_yscale("log"); axes[0].set_xlabel("epoch"); axes[0].legend(); axes[0].set_title("The
overfitting signature")
axes[1].plot(h0[:, 1], label="training accuracy"); axes[1].plot(h0[:, 3], label="validation
accuracy")
axes[1].set_xlabel("epoch"); axes[1].legend(); axes[1].set_title("Accuracy");
axes[1].set_ylim(0.5, 1.01)
plt.tight_layout(); plt.show()
print(f"training accuracy {h0[-1, 1]:.1%}, validation accuracy {h0[-1, 3]:.1%}")
print(f"validation loss: lowest {h0[best_epoch, 2]:.3f} at epoch {best_epoch}, at the end {h0[-1,
2]:.3f}")
```

```
train 200  validation 798  test 799
```




```
training accuracy 100.0%, validation accuracy 93.4%
validation loss: lowest 0.245 at epoch 37, at the end 0.309
```

This is the **overfitting signature** from the lecture:

- **training loss keeps falling** towards 0, and training accuracy hits 100% (memorised);
- **validation loss bottoms out and then rises.** The network grows ever more confident on the training images in ways that don't carry over, so on unseen images its wrong answers become more confidently wrong.

Validation **accuracy** often plateaus rather than collapsing, which is why the lecture says to **compare losses, not just training accuracy**.

Every regulariser from the lecture, on the same problem

Each setting is trained 3 times (3 random seeds) and averaged, because single runs on 200 images are noisy.

```
settings = {
    "no regularization":          dict(),
    "L2 penalty (lambda 1e-3)":   dict(l2=1e-3),
    "L1 penalty (lambda 1e-4)":   dict(l1=1e-4),
    "dropout p = 0.5":            dict(p_drop=0.5),
    "augmentation (1-px shift)":  dict(augment=True),
    "Adam + weight_decay 1e-2":   dict(weight_decay=1e-2),
    "AdamW weight_decay 1e-2":    dict(weight_decay=1e-2, optimizer="adamw"),
}
rows = []
for name, kw in settings.items():
    final_val, final_test, es_test, zeros = [], [], [], []
    for seed in range(3):
        net, h = train_reg(seed=seed, **kw)
        final_val.append(h[-1, 3]); final_test.append(evaluate(net, Xte, yte)[1])
        weights = torch.cat([m.weight.detach().flatten() for m in net if isinstance(m,
nn.Linear)])
        zeros.append((weights.abs() < 1e-3).float().mean().item())
    rows.append((name, np.mean(final_val), np.mean(final_test), np.std(final_test),
np.mean(zeros)))
print(f"{'setting':>27} {'val acc':>8} {'test acc':>14} {'weights ~0':>11}")
for name, va, te, sd, z in rows:
    print(f"{name:>27} {va:>8.1%} {te:>8.1%} ± {sd:.1%} {z:>10.0%}")
```

```
                    setting  val acc       test acc  weights ~0
         no regularization    93.4%    93.5% ± 0.3%         2%
   L2 penalty (lambda 1e-3)    93.9%    94.1% ± 0.2%        24%
   L1 penalty (lambda 1e-4)    93.7%    93.7% ± 0.2%        88%
            dropout p = 0.5    94.1%    93.9% ± 0.5%         2%
  augmentation (1-px shift)    93.1%    91.8% ± 0.6%         2%
   Adam + weight_decay 1e-2    93.6%    93.7% ± 0.2%        24%
    AdamW weight_decay 1e-2    93.5%    93.5% ± 0.3%         2%
```

Early stopping

Keep training, but remember the weights from the epoch with the **lowest validation loss**, and stop once it hasn't improved for `patience` epochs.

```
import copy
def train_early_stopping(patience=20, max_epochs=300, seed=0):
    torch.manual_seed(seed); np.random.seed(seed)
    net = big_net(); opt = torch.optim.Adam(net.parameters(), lr=1e-3)
    best_loss, best_state, best_epoch, waited = np.inf, None, 0, 0
    for epoch in range(max_epochs):
        loss = nn.functional.cross_entropy(net(Xtr), ytr); opt.zero_grad(); loss.backward();
opt.step()
        val_loss, _ = evaluate(net, Xva, yva)
        if val_loss < best_loss:
            best_loss, best_state, best_epoch, waited = val_loss,
copy.deepcopy(net.state_dict()), epoch, 0
        else:
            waited += 1
            if waited >= patience:
                break
    net.load_state_dict(best_state)
    return net, best_epoch, epoch

es_acc = []
for seed in range(3):
    net, best_ep, stopped = train_early_stopping(seed=seed)
```

```
    es_acc.append(evaluate(net, Xte, yte)[1])
    print(f"seed {seed}: best validation loss at epoch {best_ep}, stopped at epoch {stopped}
(instead of 300), test accuracy {es_acc[-1]:.1%}")
print(f"early stopping: test accuracy {np.mean(es_acc):.1%} ± {np.std(es_acc):.1%}")
```

```
seed 0: best validation loss at epoch 37, stopped at epoch 57 (instead of 300), test accuracy 93.
5%
seed 1: best validation loss at epoch 36, stopped at epoch 56 (instead of 300), test accuracy 92.
7%
seed 2: best validation loss at epoch 50, stopped at epoch 70 (instead of 300), test accuracy 93.
4%
early stopping: test accuracy 93.2% ± 0.3%
```

Honest reading of the regularization results

Before reading the tables: with 200 training images and 3 seeds, differences of about **±1 percentage point are within the noise**. Look for patterns, not for a winner.

- **L1 does exactly what the lecture says to the weights.** It drives most weights to (almost) zero, making the network **sparse**. Compare the "weights ~0" column with the others. Its effect on accuracy is modest.
- **L2 (penalty in the loss) and Adam's `weight_decay`** behave alike (both leave about 24% of the weights near zero), because with Adam, `weight_decay` is just an L2 term added to the gradient.
- **AdamW** applies **decoupled** decay directly to the weights: each step shrinks w by lr × weight_decay × w = 0.001 × 0.01 = 0.001% here, which over 300 steps is very weak. That's why its row looks like no regularization. Same number, different mechanism: the lecture's reason AdamW exists. In practice AdamW is used with larger values (e.g. 0.05–0.1) and longer training.
- **The best test results came from L2 (94.1%) and dropout (93.9%)**, both about half a point above no regularization (93.5%). That gain is real but small, about the size of the seed-to-seed noise.
- **Early stopping** gave a similar test accuracy (93.2% vs 93.5%), but stopped after about 60 epochs instead of 300 (5× cheaper) and kept the weights from the lowest validation loss.
- **Augmentation by 1-pixel shifts did not help here.** On an 8×8 digit a 1-pixel shift is a big change (12% of the width) and `torch.roll` wraps pixels around the edge. The lecture's warning applies: *"incorrect transformations can create mislabeled or unrealistic examples"*. On real-sized images, small shifts and crops are standard and usually help.
- **None of them is magic.** The model still reaches 100% training accuracy, and the gap to validation shrinks a little. The biggest single cure for overfitting is **more data**.

> **Rule from the lecture:** establish a clear **baseline** first, then add regularization one change at a time, judging on **validation** data. Look at the test set only once, at the end.

✅ Check yourself

1. Normalization or regularization: which one is mainly about optimisation, and which about generalisation? *(normalization: optimisation; regularization: generalisation)*
2. Why did gradient descent on raw features either diverge or stall? *(x2's large scale forces a tiny learning rate, at which x1 barely learns)*

3. A feature has one huge outlier. Min-max or standardisation, and why? *(standardisation, or robust scaling; min-max squashes everything else into a tiny range)*
4. You compute μ and σ on the whole dataset before splitting. What's wrong? *(data leakage: the test set influenced training, so the score is optimistic)*
5. BatchNorm vs LayerNorm: what does each average over? Which one behaves differently in train and eval mode? *(batch per feature vs features per sample; BatchNorm)*
6. What does L1 do to weights that L2 doesn't? *(drives many exactly or nearly to zero: sparsity)*
7. How do you spot overfitting from the curves? *(training loss keeps falling while validation loss stops improving or rises)*
8. Why does AdamW exist? *(with Adam, adding L2 to the loss is not the same as true weight decay; AdamW decouples the decay from the adaptive gradient step)*

DL & GenAI · Cheat Sheet (lectures 04-08 → 04-09)

History (04-08)

Year	What	Who
1943	first artificial neuron (logic with binary inputs/outputs)	McCulloch & Pitts
1958	perceptron: first **trainable** network (linearly separable only)	Rosenblatt
1969	*Perceptrons*: single-layer limits (XOR) → **AI winter**	Minsky & Papert
1986	**backpropagation** popularised → multi-layer training	Rumelhart, Hinton, Williams
1998	**LeNet-5** CNN: digits, cheques, postcodes	LeCun
2006	**Deep Belief Networks**, layer-wise pre-training → modern DL	Hinton
2012	**AlexNet** wins ImageNet: **GPUs, ReLU, Dropout**	Krizhevsky, Sutskever, Hinton
2014	**GANs** (generator vs discriminator)	Goodfellow
2017	**Transformer**, "Attention Is All You Need"	Google
2020+	foundation models & GenAI: LLMs, multimodal, diffusion (GPT, Llama, Gemini, Claude, Stable Diffusion)	many
- **Success factors:** big datasets · GPUs/TPUs · better algorithms (backprop, ReLU, BatchNorm, Dropout, attention) · open-source frameworks (TensorFlow, PyTorch, Keras) · cloud.		
- **Model evolution:** neuron → perceptron → MLP → CNN → RNN → LSTM → Transformer → foundation models.		
- **"Deep"** = number of **hidden layers** (not deep thinking). Layers learn edges → corners/textures → shapes → parts → object, **automatically**.		

AI ⊃ ML ⊃ DL ⊃ GenAI (05-08)

	AI	ML	DL	GenAI
idea	mimic intelligent behaviour	learn from data	multi-layer neural nets	generate new content
features	n/a	**manual**	**automatic**	automatic, massive scale
data	varies	moderate (1000s)	large (100k+)	massive (web-scale)
architecture	rules, search…	trees, SVM, linear	MLP, CNN, RNN/LSTM	**Transformers**, diffusion
example	rule-based chess engine	spam filter	CNN on X-rays	LLM writing an essay
- Three distinguishing factors: **data, feature engineering, architecture**.				
- Misconceptions: AI ≠ ML · DL isn't "just more layers" (automatic features) · GenAI *is* scaled DL · bigger isn't always better.				

Biological → artificial neuron

dendrites	synapse strength	soma	firing threshold	axon	plasticity
inputs x_i	weights w_i	$\Sigma w_i x_i + b$	activation $f(z)$	output	weight updates (backprop)
- Brain: ~86 billion neurons, ~100 trillion synapses, ~20 W. Artificial: numbers, synchronous steps, exact learning rule, a **drastic simplification** by design.					

Perceptron (06-08)

- **$z = \Sigma w_i x_i + b$, $\hat{y} = step(z)$** (1 if $z \geq 0$). Worked example: x = (2, 3), w = (0.5, 0.8), b = −2 → z = **1.4** → Pass.
- **Update:** $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$, $b \leftarrow b + \eta(y - \hat{y})$, error ∈ {−1, 0, +1}. Worked example: w (0.2, −0.4), b −1, y 1, ŷ 0, η 0.1 → **w (0.4, −0.1), b −0.9**.
- η too small = slow, too big = overshoot/diverge (gradient descent). Plain perceptron from zero weights: η doesn't change the epoch count.
- **Linearly separable only:** AND ✓, OR ✓, **XOR ✗** (best single line 75%) → MLP with a hidden layer (+ backprop) solves it.

Activation functions (07-08, 11-08)

f	formula	range	slope	use
Step	1 if $z \geq 0$ else 0	$\{0, 1\}$	0 (no gradient)	classic perceptron
Linear	z	$(-\infty, \infty)$	1	**regression output**
Sigmoid	$1/(1 + e^{-z})$	$(0, 1)$	$\sigma(1 - \sigma) \leq$ **0.25**	**binary output**, gates
tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	$(-1, 1)$, **zero-centred**	$1 - \tanh^2 \leq 1$	RNN/LSTM states
ReLU	$\max(0, z)$	$[0, \infty)$	1 / 0	**hidden layers** (+ He init)
Leaky ReLU	z or αz (α fixed, e.g. 0.01)	$(-\infty, \infty)$	1 / α	dying-ReLU concern
PReLU	z or az (**a learned**)	$(-\infty, \infty)$	1 / a	CNNs, adaptive slope
- No activation → stacked linear layers = **one linear layer**.				
- **Vanishing gradient:** small slopes (sigmoid/tanh saturation) multiplied over layers → early layers stop learning.				
- **Dying ReLU:** slope **exactly 0** for negative input → the neuron never updates. Leaky/PReLU keep a non-zero slope.				

Normalization & regularization (04-09)

- **Normalization = optimisation** (scale). **Regularization = generalisation** (overfitting). Complementary.
- **Standardise** $(x - \mu)/\sigma$ · **min-max** $(x - \min)/(\max - \min)$ (outlier-sensitive) · pixels/255 · fit statistics on **train only** (no leakage).
- **BatchNorm:** batch μ, $\sigma^2 \rightarrow \hat{x} = (x - \mu)/\sqrt{(\sigma^2 + \varepsilon)} \rightarrow \gamma\hat{x} + \beta$ · running statistics at inference · CNNs.
- **LayerNorm:** per-sample statistics over features · batch-independent · Transformers. Instance norm → style transfer; Group norm → small batches.
- **Overfitting:** training loss ↓ while validation loss ↑ (underfit = both poor).
- **L1** $\lambda\Sigma|w| \rightarrow$ **sparsity** · **L2** $(\lambda/2)\Sigma w^2 \rightarrow$ shrinkage (gradient + λw) · **Elastic Net** = both.
- **Dropout** p: zero activations randomly in training, scale survivors by $1/(1 - p)$, off at inference.
- **Early stopping:** best validation epoch + patience · **augmentation:** label-preserving variations · **weight decay:** $(1 - \eta\lambda)w - \eta\nabla L$; with Adam use **AdamW** (decoupled).

- Guide: different scales → standardise · CNN → BatchNorm · Transformer → LayerNorm · overfit → L2 / dropout / early stopping · sparse → L1.